

PIMphony: Overcoming Bandwidth and Capacity Inefficiency in PIM-based Long-Context LLM Inference System

Hyucksung Kwon^{*1}, Kyungmo Koo^{*1}, Janghyeon Kim¹, Woongkyu Lee¹, Minjae Lee¹, Gyeonggeun Jung², Hyungdeok Lee³, Yousub Jung³, Jaehan Park³, Yosub Song³, Byeongsu Yang³, Haerang Choi³, Guhyun Kim³, Jongsoon Won³, Woojae Shin³, Changhyun Kim³, Gyeongcheol Shin³, Yongkee Kwon³, Ilkon Kim³, Euicheol Lim³, John Kim², Jungwook Choi^{‡1}

¹Hanyang University, Seoul, Republic of Korea,

²KAIST, Daejeon, Republic of Korea,

³Solution Advanced Technology, SK hynix, Republic of Korea

{momarom, kookyungmo, kkt20, lwghanyang, Imj4666, choij}@hanyang.ac.kr

gyeonggeun@kaist.ac.kr, jjk12@kaist.edu

{hyungdeok.lee, ryan.song, jaehan3.park, yosub.song, byeongsu.yang, haerang.choi, guhyun.kim, jongsoon.won, woojae.shin, changhyun4.kim, gyeongcheol.shin, yongkee.kwon, ilkon.kim, euicheol.lim}@sk.com

Abstract—The expansion of long-context Large Language Models (LLMs) creates significant memory system challenges. While Processing-in-Memory (PIM) is a promising accelerator, we identify that it suffers from critical inefficiencies when scaled to long contexts: severe channel underutilization, performance-limiting I/O bottlenecks, and massive memory waste from static KV cache management. In this work, we propose PIMphony, a PIM *orchestrator* that systematically resolves these issues with three co-designed techniques. First, *Token-Centric PIM Partitioning (TCP)* ensures high channel utilization regardless of batch size. Second, *Dynamic PIM Command Scheduling (DCS)* mitigates the I/O bottleneck by overlapping data movement and computation. Finally, a *Dynamic PIM Access (DPA)* controller enables dynamic memory management to eliminate static memory waste. Implemented via an MLIR-based compiler and evaluated on a cycle-accurate simulator, PIMphony significantly improves throughput for long-context LLM inference (up to 72B parameters and 1M context length). Our evaluations show performance boosts of up to 11.3× on PIM-only systems and 8.4× on xPU+PIM systems, enabling more efficient deployment of LLMs in real-world long-context applications.

I. INTRODUCTION

The rapid advancement of Large Language Models (LLMs) has transformed fields ranging from natural language processing to intelligent agents by enabling the generation of contextually relevant responses across diverse applications [9], [10], [15], [26], [53]. In particular, long-context LLMs, capable of maintaining coherence across hundreds of thousands of tokens, have significantly enhanced contextual relevance in various tasks. For instance, long document summarization [74] generates cohesive summaries from dispersed information across different sections of extensive text, while repository-level code analysis [47] extends programming assistants' capabilities to analyze entire codebases comprising thousands of lines. Furthermore, chain-of-thought (CoT) reasoning has recently

improved answer quality by leveraging multi-step contextual reasoning [9], [50], [52], [70].

As these LLMs continue to expand — some models exceeding 1T parameters and context windows of 1M tokens [48], [65], [66] — the memory bandwidth and capacity are known to be the bottlenecks of overall system performance [16], [57]. Conventional accelerators such as multi-GPUs suffer from poor compute utilization since large matrix-vector (GEMV) operations with low compute intensity dominate these autoregressive decoding phases [22]. Recent work such as FlashInfer [71] and others [1]–[3] propose different frameworks to accelerate LLM on GPU-based systems through token-level parallel processing or dynamic batch-size optimization; however, their performance benefits are still bounded by GPU's limited memory bandwidth and capacity.

Recently, Processing-in-Memory (PIM) [5], [19], [36], [51] has been proposed to accelerate LLM by exploiting high internal memory bandwidth [16], [21], [54]. NeuPIMs [21] proposes an NPU+PIM heterogeneous system where NPU is leveraged for compute-intensive kernels (i.e., GEMM) while PIM is leveraged for memory-bound kernels (i.e., GEMV) to accelerate LLM inference. PIM-only systems (e.g., CENT [16]) have also been proposed as a multi-PIM node system to handle large LLMs via CXL-based memory expansion. While these prior works accelerate LLM inference with PIM, they primarily focus on relatively small input context lengths (e.g., 4K¹). In this work, we revisit PIM for LLM inference but focus on long-context LLM (up to 1M tokens) and identify how PIM inefficiency becomes problematic as the Attention layers in decoding become a greater bottleneck with longer

¹CENT does provide ablation study where they scale to 32K but the benefits of PIM decrease as sequence length increases. More importantly, this work identifies PIM inefficiency as the context length increases and maximizes overall performance from PIM.

^{*}Equal contribution. [‡]Corresponding author.

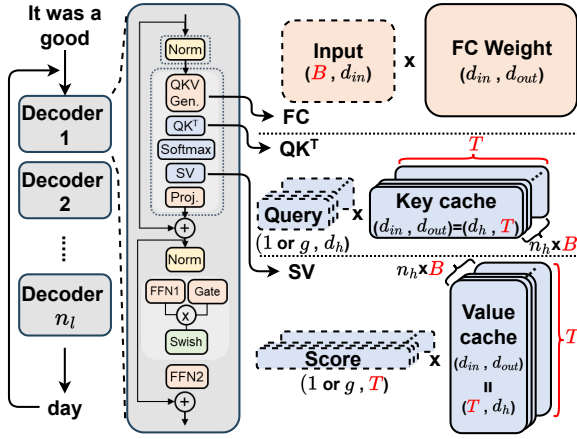


Fig. 1: Decoding Computation for Long-Context LLM (g : group size of GQA [4])

TABLE I: LLM specification and the context window (CW).

Model	n_l	n_h	d_h	$d_{in/out}$	GQA	Reference	CW
LLM-7B	32	32	128	4K-12K	×	QWEN1.5-7B [6]	32K
				4K-12K	✓ ($g = 4$)	Llama3.1-8B [13]	128K
				8K-24K	×	Qwen1.5-72B [6]	32K
LLM-72B	80	64	128	8K-24K	✓ ($g = 8$)	Llama3.1-70B [13]	128K

context. In long-context scenarios, PIM inefficiency arises from workload imbalance that leaves channels underutilized and from a fixed write–compute–read pipeline that creates I/O bottlenecks and stalls processing units. KV cache remains problematic in terms of PIM memory capacity but *static* memory allocation based on maximum context length becomes problematic when context length can vary significantly across workloads [7], [45], [73].

To address these limitations, we propose PIMphony, a PIM *orchestrator* that enables efficient data mapping and movement to improve PIM utilization through three co-designed techniques. First, unlike the conventional head-first or batch-dimension partitioning approach, *Token-Centric PIM Partitioning* (TCP) distributes tokens across all channels within a single PIM module to ensure high utilization and load-balancing regardless of batch size. Second, to mitigate the PIM I/O bottleneck, *Dynamic PIM Command Scheduling* (DCS) issues commands based on real-time data dependencies to maximally overlap computation and data movement—a capability absent in static PIM schedulers. Finally, *Dynamic PIM Access* (DPA) overcomes the limitations of static memory management; by embedding loop bounds and operand modifications into the command stream, it enables dynamic KV cache allocation to improve memory utilization. As a result, PIMphony achieves higher efficiency by better exploiting the high internal bandwidth of each PIM channel by balancing workloads, continuously feeding data to its MAC units, and allocating memory on-demand.

We implement PIMphony by extending an MLIR-based compiler and runtime to generate the PIM commands for both token-level partitioning and dynamic KV cache allocation. We developed custom compiler passes that detect transformer decoder patterns, generate PIM commands with

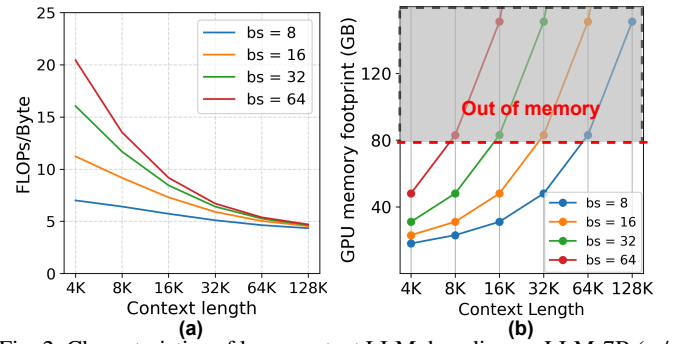


Fig. 2: Characteristics of long-context LLM decoding on LLM-7B (w/ GQA). (a) Compute intensity (FLOPs/Byte) decreases with context length. (b) GPU memory footprint grows with both context length and batch size; the dashed line marks the A100-80GB capacity.

TABLE II: Statistics of input context length.

Statistic	LongBench [7]		LV-Eval [73]	
	QMSum	Musique	multifieldqa	Loogle-SD
mean	13,966	16,362	60,780	50,693
std	6,182	1,651	31,025	26,506
max	30,456	17,917	119,480	109,221
min	2,651	6,820	20,333	13,347

dynamic row/column indices, and handle the token-parallel mapping within each module. We also modified a cycle-accurate DRAM simulator to model PIM I/O output buffering and apply PIMphony to both a heterogeneous architecture (i.e., NeuPIMs [21]) and a PIM-only architecture (i.e., CENT [16]). Across representative long-context LLM workloads, our evaluations show up to 11.3 $\times$ improvement in performance compared to state-of-the-art PIM accelerators for LLM inference.

II. BACKGROUND/MOTIVATION

A. Long-Context LLM Inference/Decoding

Long-context LLMs are built on the Transformer decoder architecture [68], which, as shown in Fig. 1, consists of n_l layers, each containing a Multi-Head Attention (MHA) module and a Feed-Forward Network (FFN). Within each of the n_h attention heads, per-head feature dimension (d_h), matrix dimension (d_{in}, d_{out}) for weight parameters, Query/Key/Value vectors ($Q, K, V \in \mathbb{R}^{d_h}$) are generated, and the K/V vectors are appended to a KV cache for all T tokens. To improve efficiency, some models employ architectural variants like Grouped-Query Attention (GQA) [4], where multiple query heads share a single set of Key and Value vectors per group. The subsequent Attention operations, QK^T and SV , access this cache. Representative LLM configurations, including GQA variants, are detailed in Table I, and benchmark characteristics are outlined in Table II.

Our analysis of long-context LLM workloads reveals memory bandwidth and capacity bottlenecks from Attention during decoding. As context length grows, compute intensity (OPs/Byte) drops sharply (Fig. 2(a)) as computation shifts from compute-intensive matrix-matrix (GEMM) to memory-bound matrix-vector (GEMV) operations for Attention—making performance heavily dependent on raw memory

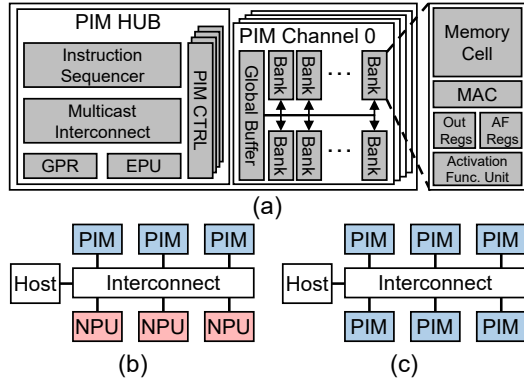


Fig. 3: Overview of PIM module/node configuration. (a): PIM module architecture. (b) and (c): PIM node configuration - heterogeneous xPU+PIM and PIM-only.

bandwidth. This bandwidth pressure is compounded by capacity demand of the KV cache. Because the KV cache increases proportionally with context length and batch size, it dominates the growth of overall memory requirements and thus imposes significant capacity pressure in long-context LLM inference (Fig. 2(b)). In this work, we exploit a multi-node PIM system to provide both the required memory capacity and high internal memory bandwidth.

B. PIM Architecture and Instruction Execution

DRAM-based Processing-in-Memory (PIM) systems [33], [40] accelerate memory-bound GEMV operations by integrating computation directly into memory. As detailed in Fig. 3(a), a PIM module integrates vector MAC units within each DRAM bank, a shared Global Buffer (GBuf) for inputs, and Output Registers (OutRegs) for results, all orchestrated by a PIM Controller. An Activation Function Unit, with an associated Activation Function Registers, is also included to support non-linear activation functions via Look-Up Table (LUT) approximations. An Extra Processing Unit (EPU) performs auxiliary operations (e.g., Softmax), while a General-Purpose Register file (GPR) stores inputs/outputs (or intermediate results) of the computation. In operation, the PIM HUB receives compiler-generated PIM instructions. The Instruction Sequencer expands each instruction by unrolling its specified repetitions and forwards the resulting instruction stream to the Multicast Interconnect. The Multicast Interconnect then decodes these instructions into channel-specific PIM commands, multicasts them to the target channels, and routes the associated data to the corresponding PIM Controllers.

Each PIM instruction carries a set of arguments that determine how it is expanded and dispatched as PIM commands (Table III). Specifically, *Ch-mask* specifies the target PIM channels, *Op-size* determines repetition count, and *GPR-addr* provides the base access address when the instruction involves data movement between the GPR and the PIM (e.g., WR-INP or RD-OUT). The Instruction Sequencer uses *Op-size* to unroll a single instruction into a sequence of repeated instructions, which are subsequently decoded by the Multicast Interconnect

TABLE III: PIM instructions for LLM inference. *Ch-mask*, *Op-size*, *GPR-addr* guide the decoding of each instruction into channel-specific PIM commands, while *GBuf-Idx*, *Out-Idx*, *Row/Col* are used during PIM channel operation.

Instruction	Description	Arguments
WR-INP	Copy Input from GPR to GBuf	Ch-mask Op-size GPR-addr GBuf-Idx
MAC	Dot-Product on a DRAM row	Ch-mask Op-size GBuf-Idx Row/Col Out-Idx
RD-OUT	Copy Output from OutReg to GPR	Ch-mask Op-size GPR-addr Out-Idx

into channel-specific PIM commands accessing consecutive addresses (e.g., GPR-addr, GBuf-Idx, or column addresses).

C. Multi-Node DRAM-PIM System

To accommodate large models, multiple nodes can be deployed in heterogeneous (xPU+PIM) or PIM-only configurations (Fig. 3(b,c)), leveraging model parallelism techniques such as Tensor Parallelism (TP) [60] and Pipeline Parallelism (PP) [23]. In TP, model parameters are partitioned across PIM modules, and parallel computation is performed on different tensor shards, with synchronization required to aggregate partial results. In contrast, PP partitions the model into transformer layers that are executed across PIM modules, allowing different micro-batches to be processed concurrently in a pipelined manner. Both approaches enable large transformer models to be mapped across multiple PIM modules by exposing parallelism at different granularities.

D. Challenges

Although PIM systems offload memory-bound Attention operations, their efficiency drops sharply in long-context scenarios. We identify three fundamental limitations that create a critical performance bottleneck:

- 1) **Channel Underutilization.** Prior PIM systems partition the KV cache by the head and batch dimensions to parallelize Attention across PIM channels [16], [21], [54]. However, this strategy is highly inefficient for long contexts. It leads to severe channel underutilization, either from workload imbalance when requests have different context lengths or from insufficient sub-batch sizes to keep all pipeline stages full.
- 2) **I/O Bottleneck.** Each dot-product in PIM follows a fixed command pipeline of WR-INP $\rightarrow$ MAC $\rightarrow$ RD-OUT (Table III), but limited I/O buffers—2KB input per channel and 4-byte output per bank [40]—frequently stall the MAC units. This issue is amplified in Attention layers, where the KV cache has a small feature dimension (d_h ; Fig. 1), limiting input/output reuse and increasing buffer turnover. Critically, static command scheduling fails to overlap data movement with computation, resulting in low MAC utilization even under optimal partitioning.
- 3) **Static Memory Management.** To handle variable context lengths, existing PIM systems pre-allocate a KV cache sized for the maximum context T_{max} , bounding the batch size to a worst-case capacity. Given that real-world workloads have diverse token lengths (Table II),

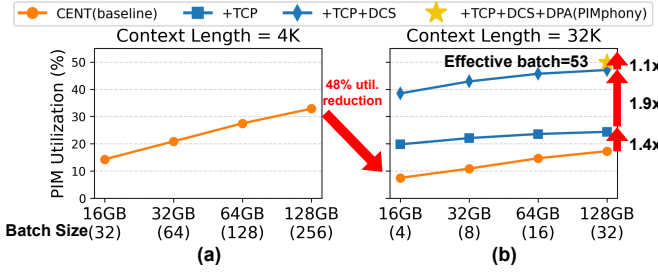


Fig. 4: PIM utilization under (a) short(4K) and (b) long(32K) contexts using CENT [16] and PIMphony on LLM-7B-32K-GQA. Batch size scales inversely with context length due to the capacity constraint.

this static approach leads to severe memory inefficiency, with an observed average capacity utilization of only 36.2% (See Sec. VIII-C). The root cause is that PIM commands embed fixed physical addresses, making it impossible to repurpose these large, unused memory regions at runtime.

Taken together, the challenges of channel underutilization, I/O bottlenecks, and static memory waste show that conventional PIM systems are fundamentally underutilized for long-context LLM inference. This inefficiency is substantial; our analysis reveals that MAC unit utilization drops by 48% at a 32K context length (Fig. 4). Such systemic limitations cannot be solved with incremental optimizations, necessitating a holistic redesign of the PIM software and hardware stack—motivating the novel orchestrator we propose.

III. PIMPHONY OVERVIEW

We propose PIMphony, a PIM *orchestrator* that systematically resolves these inefficiencies to enable high performance for long-context LLM inference. PIMphony improves PIM utilization—a critical yet previously underexplored bottleneck—through three co-designed techniques (Fig. 5).

Token-Centric PIM Partitioning (TCP). To combat channel underutilization, PIMphony introduces TCP. Unlike conventional methods that rely on the volatile batch dimension, TCP reorients parallelism along the plentiful *token* axis. By distributing token-level work across all channels, TCP decouples performance from batch size, mitigating imbalance and ensuring consistently high channel utilization.

Dynamic PIM Command Scheduling (DCS). To eliminate the I/O bottleneck, PIMphony employs DCS, which enhances the PIM controller with the ability to issue commands out-of-order based on real-time data dependencies. This dynamic approach, impossible for rigid static schedulers, effectively hides I/O latency by overlapping data movement and computation, thus maximizing MAC pipeline throughput.

Dynamic Memory Management with Dynamic PIM Access (DPA). To address wasteful static memory allocation, PIMphony features DPA, which includes a novel on-module dispatcher, which effectively acts as a lightweight, pseudo-Memory Management Unit (MMU) for the PIM. This dispatcher enables runtime virtual-to-physical address translation, allowing for dynamic, on-demand memory allocation for the KV cache. This breaks free from the static pre-allocation

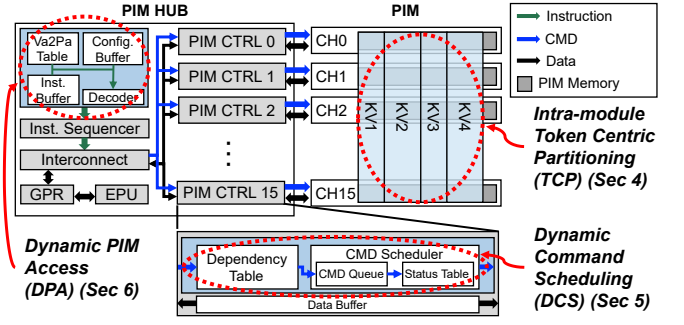


Fig. 5: High-level overview of PIMphony with the three main components highlighted.

model based on maximum context length and improves effective memory capacity utilization.

Together, TCP, DCS, and DPA form a cohesive orchestration framework that addresses PIM inefficiencies across parallelism, command scheduling, and memory management. Fig. 4(b) provides a preview of their cumulative impact on PIM utilization. Building on this architectural overview, the following sections describe each technique in detail, explaining how they jointly enable efficient long-context LLM inference.

IV. TOKEN-CENTRIC PIM PARTITIONING

To understand the limitations of existing PIM workload partitioning strategies for long-context LLM inference, we first examine a simplified example that illustrates their impact on channel utilization. Fig. 6(a) visualizes a Transformer example workload with two attention heads, two layers, and a batch size of two, distributed across two PIM modules, each containing four independently operating channels. Computation tasks are labeled with the notation $R(r, h, l)$, where r denotes the request ID, h the head index, and l the layer index, while active and inactive KV cache regions are shown in blue and shade, respectively. This example reflects long-context conditions, where a request typically consumes nearly the entire memory capacity of a single PIM channel, limiting each channel to serve only one request at a time. This example allows us to compare the effects of tensor parallelism (TP) and pipeline parallelism (PP) on channel activity, contrasting conventional PIM partitioning with our proposed token-centric PIM partitioning (TCP) approach.

A. Channel Underutilization Issue

Modern PIM-based LLM accelerators [16], [21], [54] suffer severe channel underutilization in long-context inference because existing KV cache partitioning schemes do not scale with growing context length. To distribute workload across channels within a PIM module, prior systems commonly adopt Head-First Partitioning (HFP), which assigns head-batch pairs to individual PIM channels for concurrent execution. HFP implicitly assumes the availability of sufficient head-batch parallelism to populate all channels. However, as context length increases, each request's KV cache footprint expands while the number of simultaneous requests (i.e., batch size) shrinks. In the extreme, a single long-context request can

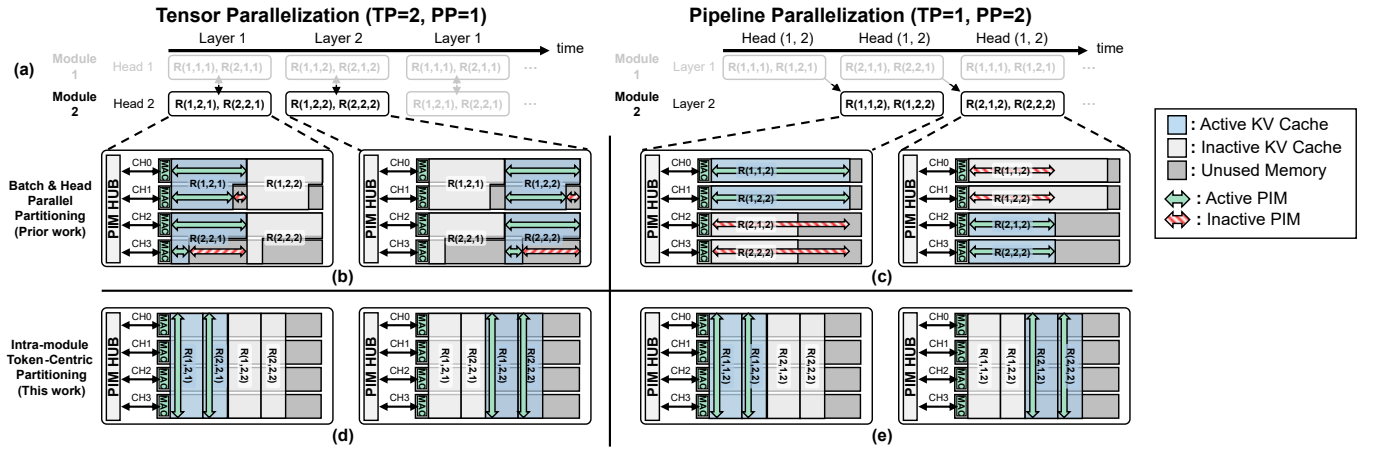


Fig. 6: Comparison of KV cache partitioning strategies across PIM channels for tensor parallelism (TP) (b,d) and pipeline parallelism (PP) (c,e). Prior approach [16], [21], [54] (Head/Batch-First Partitioning (HFP)) is shown in (b,c) while the proposed Token-Centric Partitioning (TCP) is shown in (d,e). The example highlights the inefficient utilization of HFP across the PIM channels, as not all channels can be utilized simultaneously. For clarity, only one PIM module with four channels is shown. $R(r, h, l)$ indicates Request r , Head h , and Layer l .

occupy an entire channel, drastically reducing the number of available head-batch tiles. As a result, HFP fails to sustain high channel utilization in long-context settings, leaving many channel-level MAC units idle despite abundant token-level parallelism. In the following, we analyze how this limitation of HFP manifests under tensor and pipeline parallelism.

B. Analysis of HFP under Tensor and Pipeline Parallelism

Tensor parallelism (TP) and pipeline parallelism (PP) organize LLM execution across PIM modules at different granularities—TP partitions attention heads across modules, while PP distributes consecutive layers. Under both schemes, however, the intra-module channel allocation remains governed by head-first partitioning (HFP), which ultimately determines how workloads are mapped to individual PIM channels.

Load Imbalance from HFP under TP. Under TP, HFP leads to load imbalance when requests have different token lengths, leading to varied execution times across channels. Channels assigned to shorter sequences finish their computations earlier and must wait for those processing longer sequences, limiting the overall throughput to that of the slowest channel and reducing effective parallelism. For example, in Fig. 6(b), Channel 2 and 3 are assigned to Request 2 with a shorter token length and thus performs less computation, becoming idle while other channels continue execution. Although short-context systems can mitigate such imbalance by assigning multiple requests per channel for load balancing (e.g., NeuPIMs [21]), this approach becomes infeasible in long-context inference, where a single request can fully occupy a channel’s memory capacity.

Sparse Channel Activation from HFP under PP. Under PP, HFP activates only the subset of channels associated with the request assigned to each pipeline stage. In long-context scenarios, where a single request can occupy an entire channel, this results in sparsely populated stages with many idle channels. As illustrated in Fig. 6(c), only a fraction of the available channels are active at any given time. This stage-

level idling, compounded by pipeline bubbles that form when subsequent stages are empty, results in persistent underutilization—a problem especially pronounced with the limited batch sizes of long-context workloads.

C. Intra-Module Token-Centric Partitioning

To overcome the channel underutilization inherent in HFP, we introduce Token-Centric PIM Partitioning (TCP). TCP partitions the token dimension of a single head across all available PIM channels, enabling token-level parallelism within each PIM module. In QK^T operations, each channel processes a distinct segment of tokens concurrently, enabling parallel computation across the module. In SV operations, each channel performs token-wise partial reduction over its assigned tokens, and the partial results are reduced through the shared PIM HUB and GPR to produce the final output. For example, in Fig. 6, assume each channel contains 16 banks, with a total token length of 16K and a head dimension of 32. Under this configuration, QK^T assigns 4K tokens to each channel for parallel computation, while SV assigns 2K tokens per channel and performs a single global reduction across channels through the PIM HUB to generate the final result.

TCP enables full channel activation in long-context inference, where the token sequence length is sufficiently large to keep all processing units active. As illustrated in Fig. 6(d) and (e), TCP distributes token computations across channels, ensuring full channel activation. Under TP, TCP mitigates token-length imbalance across requests that previously caused idle channels, while under PP, TCP allows all channels to participate concurrently regardless of the active pipeline stage. Under a commercial PIM module [62] configuration with 16 channels and 16 banks per channel, full channel activation is achieved once the token length exceeds 256 for QKT and 32 for SV.

Under TCP, each PIM channel produces partial outputs that must be combined within the module to form a complete result. This aggregation step operates at the inter-channel level,

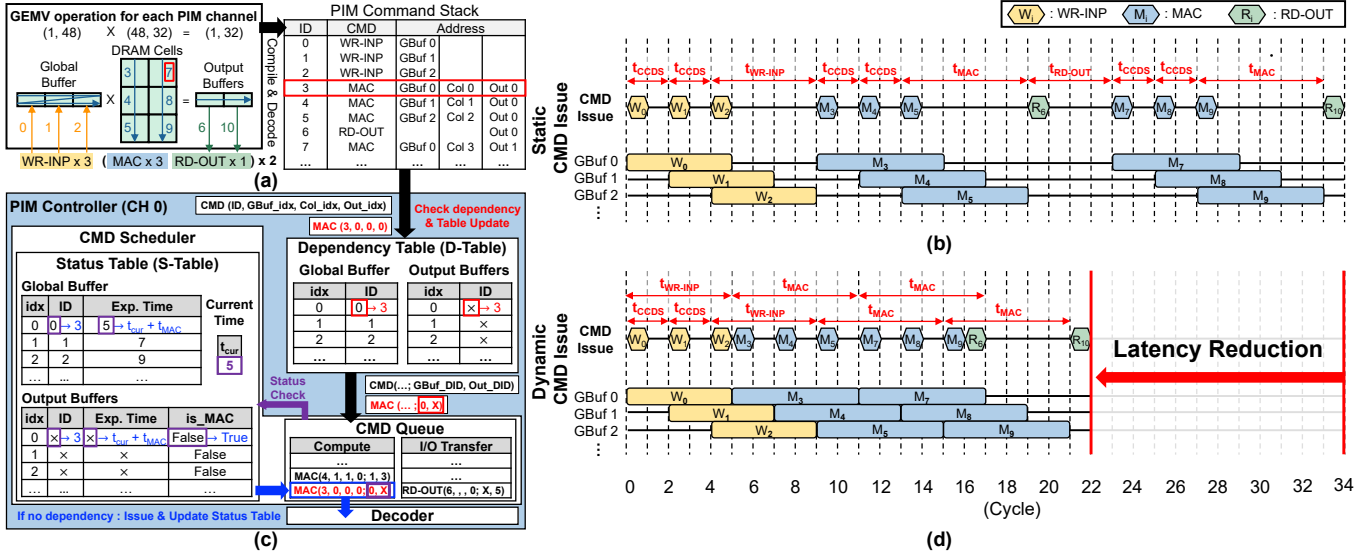


Fig. 7: Dynamic PIM Command Scheduling. (a) A GEMV operation example and its command stack, (b) timing diagram for baseline PIM command schedule, (c) detailed Dynamic PIM Command Scheduling (DCS) example within the PIM controller, and (d) resulting timing diagram after DCS where MAC instructions can be executed in advance. In (b)&(d), Each GB 0/1/2 denotes an entry of a Global Buffer.

where outputs from different channels are gathered through the general-purpose registers (GPR) at the PIM HUB and finalized by the Extra Processing Unit (EPU) (Fig. 3(a)). For QK^T , the aggregation after matrix multiplication involves only concatenation during the subsequent Softmax in the EPU, incurring no measurable latency. Meanwhile, SV performs a single inter-channel reduction per module, whose cost is minimal—below 0.2% of total attention latency for an LLM-7B with 16K tokens. Because TCP partitions tokens only within a module, it avoids inter-module synchronization, keeping aggregation and synchronization overhead negligible.

V. DYNAMIC PIM COMMAND SCHEDULING

A. PIM Command Execution Overview

Modern PIM architectures follow a command-driven execution model where primitive operations—WR-INP, MAC, and RD-OUT—are issued sequentially to hardware units (see Table III). WR-INP writes a 32B tile to a Global Buffer (GBuf) entry, MAC reads that entry for multiplication and accumulates results into per-bank OutRegs and RD-OUT drains a 2B result from all 16 banks concurrently (32B in total). These primitives are composed into a command stack to perform computation. For example, the FP16 GEMV in Fig. 7(a) is executed by streaming the input tiles via WR-INP, accumulating partial dot products with MAC, and retrieving the output tiles with RD-OUT. Due to the pipelined operation of the data bus, transferring 32B tiles has a minimum command-to-command interval (t_{CCDS}). As illustrated in Fig. 7(b), successive WR-INPs (W_0, W_1, W_2) are issued t_{CCDS} cycles apart, pipelining the streaming of input tiles.

Conventional PIM controllers [37], [62] execute commands using *static scheduling*: the controller issues commands strictly in-order in a fixed WR-INP $\rightarrow$ MAC $\rightarrow$ RD-OUT pattern (as generated from the instruction stream). To avoid data hazards,

it decides issue by enforcing time gaps derived from fixed command execution times (t_{WR-INP} , t_{MAC} , t_{RD-OUT}) between commands. For example, a MAC must wait at least t_{WR-INP} after the preceding WR-INP to ensure the input tile is fully written into the GBuf. As shown in Fig. 7(b), this approach needlessly serializes operations, causing pipeline stalls even when no true data dependency exists between commands (e.g., the input write W_2 and the computation M_3 , or the output read R_6 and computation M_7).

B. I/O Bottleneck Analysis

Attention layers inherently incur frequent I/O transfers (WR-INP, RD-OUT) due to their significantly lower data reuse compared to fully-connected (FC) layers. This effect is particularly pronounced in Attention's core operations: in QK^T , a small input dimension (d_{in}) reduces output reuse, while in SV, a small output dimension (d_{out}) limits input reuse. In both cases, the reduced data reuse necessitates more frequent I/O transfers. Moreover, under conventional PIM controllers, these frequent transfers are executed with *static scheduling*, which can further amplify the resulting performance bottleneck.

Static scheduling enforces a fixed command ordering and timing without tracking per-entry data dependencies across GBuf and OutRegs. Consequently, it fails to overlap data transfer and computation, serializing them even when data hazards are cleared. This conservative scheduling blocks MAC execution even when resources are available, leading to substantial pipeline penalties and idle cycles. Fig. 8 quantifies the impact of this inefficiency, showing that as matrix dimensions (d_{in} , d_{out}) decrease, pipeline stall and I/O transfer time become dominant. In particular, for small dimensions typical of Attention (128, corresponding to a head dimension), MAC utilization drops sharply to 14.7%. This trend indicates that frequent I/O further exacerbates the limitations of static scheduling.

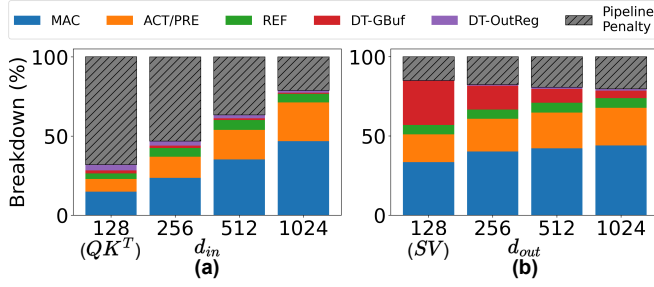


Fig. 8: Latency breakdown across matrix dimensions. MAC is computation time; ACT/PRE and REF are DRAM activation/precharge and refresh time; DT-GBuf and DT-OutReg represent I/O transfers time; Pipeline Penalty captures cumulative stalls across PIM commands.

Since frequent I/O is unavoidable in Attention workloads, this fundamental limitation of *static scheduling* motivates the need for a more flexible, command-level, dependency-aware Dynamic Command Scheduling (DCS).

C. Dynamic PIM Command Scheduling

I/O-aware Buffering. To mitigate the bottleneck from frequent I/O transfer, we introduce I/O-aware buffering that decouples I/O transfers from computation, allowing the controller to issue independent PIM commands more aggressively and keep the pipeline busy. For this, we repurpose the existing multi-entry GBuf as a dedicated input buffer and expand the Output Registers into larger Output Buffers (OBuf) for output staging. Both of these buffers are implemented as dual-port memory, enabling concurrent accesses: while MAC consumes the current inputs and accumulates partial sums, the controller can prefetch the next inputs into GBuf or drain completed outputs from OBuf, increasing I/O–compute overlap.

Dynamic Command Scheduling. We extend the PIM controller with a dynamic scheduling mechanism that enforces inter-command timing constraints only when true data dependencies exist between I/O transfer and MAC operations. To support this, we introduce two hardware structures—the Dependency Table (D-Table) and the Status Table (S-Table)—as illustrated in Fig. 7(c).

Each command is assigned a unique identifier (ID) and maintains a Dependency ID (DID), which specifies the ID of the command that must complete before the current command can be safely executed. The D-Table enables this dependency assignment by recording, for each GBuf and OBuf entry, the most recent command that accessed it. When a new command arrives, the controller consults the corresponding D-Table entry, assigns the recorded command ID as the new command’s DID, and updates the D-Table with the new command ID.

The S-Table maintains the execution status to determine whether a dependency has been resolved. For each GBuf and OBuf entry, it records the ID of the most recent accessing command along with the cycle at which that access completes (i.e., an expiration timestamp). In addition, the S-Table maintains an *is-MAC* flag for OBuf entries to identify consecutive MAC operations targeting the same buffer location, enabling pipelined execution without unnecessary stalls.

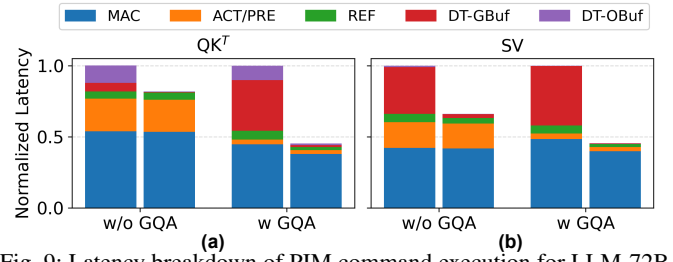


Fig. 9: Latency breakdown of PIM command execution for LLM-72B Attention. (a) QK^T , (b) SV . Each bar compares execution without (left) and with DCS (right). Both employ the *row-reuse mapping*

Together, these structures govern command scheduling and dispatch. After dependency assignment via the D-Table, the controller classifies each command and enqueues it into either the I/O transfer queue or the compute queue, allowing out-of-order execution between I/O transfer commands and MAC commands while preserving in-order execution within each queue. For each command considered for execution, the controller consults the S-Table to determine its execution readiness. A command is dispatched only (i) if the S-Table entry corresponding to its accessed buffer reports a matching command ID to the command’s DID and (ii) the current cycle t_{cur} exceeds the recorded expiration time, ensuring that the predecessor command has completed. When these conditions are satisfied, the controller issues the command and updates the S-Table to reflect the new access and its completion time. For consecutive MAC operations accessing the same OBuf entry, the *is-MAC* flag allows the controller to bypass the t_{MAC} gap and issue them at the minimum t_{CCDS} interval.

PIM Controller Example. Fig. 7(c) illustrates an example of the PIM controller operation. When command M_3 arrives, the controller consults the D-Table and identifies that command W_0 was the most recent command to access GBuf entry 0, which M_3 targets. Accordingly, M_3 is assigned a GBuf Dependency ID of 0. This dependency association indicates that M_3 must wait for W_0 to complete before accessing the GBuf entry. To determine whether this dependency has been resolved, the controller examines the S-Table entry corresponding to GBuf index 0. Issuance of M_3 is permitted only if the command ID recorded in the S-Table matches M_3 ’s GBuf-DID and the current cycle t_{cur} exceeds the recorded expiration timestamp. This check ensures that M_3 is issued only after W_0 has completed its access.

DCS’s Latency Saving. DCS reduces latency by dynamically relaxing inter-command timing as dependencies permit, thereby increasing pipeline efficiency and enabling out-of-order issue of independent commands. Fig. 7(d) provides an example where W_0 completes, the scheduler immediately issues M_3 because it is independent of W_2 , even though W_2 has not yet finished (relaxing inter-command timing). This allows computation to proceed without waiting unnecessarily. Similarly, M_7 is issued before R_6 because they have no data dependency, which permits M_7 to directly follow M_5 with only a minimal t_{CCDS} interval (out-of-order execution). By bypassing such stalls, the DCS scheduler improves MAC

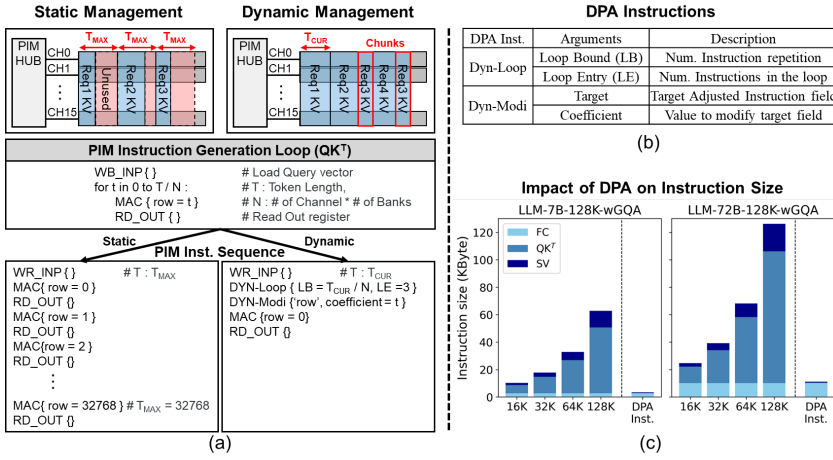


Fig. 10: (a) Limitations of static memory management in PIM where memory is allocated based on max token size, (b) new DPA Instructions introduced with PIMphony, and (c) impact of DPA on instruction size.

pipeline throughput and significantly reduces total execution time. In the example shown in Fig. 7(d), DCS reduces the execution from 34 cycles to just 22 cycles.

Enabling KV Cache Reuse in GQA. When operating across multiple DRAM rows, completing all computations on the currently opened row before moving to the next reduces ACT/PRE overhead; we refer to this as a *row-reuse mapping*. When applied to GQA [4], this means processing all inputs (queries/scores) that share the row-resident KV cache before switching DRAM rows. However, this mapping increases I/O transfer because queries/scores are frequently swapped into the GBuf, exacerbating pipeline stalls under *static scheduling*. As shown in Fig. 9, this raises DT-GBuf overhead even with weight reuse. DCS mitigates the issue by overlapping I/O transfers with MAC execution, reducing pipeline stalls and allowing *row-reuse mapping* to yield performance gains.

VI. DYNAMIC PIM MEMORY MANAGEMENT

Managing memory in long-context LLM decoding is challenging due to variability in context lengths. While GPU systems [34] have implemented dynamic memory allocation to handle such variability, existing PIM architectures [16], [32], [42] remain fundamentally constrained by static control mechanisms. Conventional PIMs rely on a pre-compiled instruction sequence—a set of instructions used to execute a target operation—with fixed loop counts and operand addresses. This control scheme can handle repetitive computations, such as layer or head loops, whose iteration counts remain constant during inference. However, it cannot adapt to the variable-sized attention computation dictated by the current token length. Because all PIM instructions are statically compiled, the corresponding KV cache addresses must be pre-assigned at compile time, thereby preventing dynamic memory management during decoding. Thus, conventional PIMs do not provide the on-demand allocation required by vLLM’s paged-attention [34], leading to inefficient memory utilization.

To overcome this limitation, we introduce the *Dynamic PIM Access* (DPA) controller, which effectively acts as a

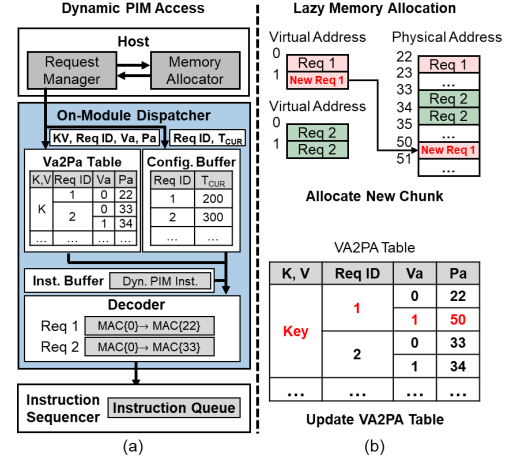


Fig. 11: To enable dynamic memory management with DPA, high-level overview of (a) on-module dispatcher and (b) lazy memory allocation.

lightweight, pseudo-Memory Management Unit (MMU) for PIM. DPA supports diverse KV cache sizes and enables “lazy” memory allocation by providing a mechanism for runtime virtual-to-physical address translation—a capability absent in conventional PIM architectures.

A. Conventional KV Cache Management in PIM

PIM-based systems accelerate Attention computations by managing the KV cache directly within their memory modules. However, existing systems rely on an inefficient static memory management scheme, as illustrated in Fig. 10(a). To handle the worst-case scenario, memory is preallocated based on the maximum possible token length. This leads to severe memory capacity underutilization when the actual token length is shorter than the reserved maximum, a common occurrence in dynamic long-context workloads.

The root cause of this underutilization is a fundamental architectural limitation: conventional PIM [62] designs lack explicit support for control instructions that enable runtime-dependent operand or address computation. While some architectures (e.g. [42]) provide limited loop constructs via JUMP instructions, these loops can only iterate over statically pre-compiled physical memory addresses. As a result, even within a loop, memory accesses cannot be adjusted based on the actual token length observed at runtime. Consequently, the compiler is forced to conservatively generate instruction sequences that assume the worst-case maximum token length, which enforces rigid static memory allocation and prevents the system from adapting to dynamic context sizes during LLM decoding (Fig. 10(a)).

To overcome the limitations of static PIM memory, we leverage two key properties of the Attention computation structure. First, the instruction sequence exhibits a highly repetitive compute pattern (Fig. 10(a)), allowing it to be expressed as a compact loop instead of a long, static list. Second, and more critically, *the instruction operands are token-length-dependent*. For example, the MAC row index is calculated directly from the current token length (T_{cur}) using

the formula, $row = T_{cur} / (n_{CH} * n_{Bank})$. This enables the use of logical, loop-dependent virtual addresses (VA), which are translated to physical addresses (PA) at runtime using a VA2PA table.

B. Dynamic PIM Access (DPA) Instructions

Based on the insights of Attention’s computational structure, we introduce Dynamic PIM Access (DPA) instructions, a flexible instruction set designed to avoid the static execution model of conventional PIMs. As outlined in Fig. 10(b), DPA instructions empower the PIM to handle dynamic workloads by encapsulating repetitive token operations into a compact, runtime-executable format. This is achieved through two key instructions:

- **Dyn-Loop** instruction encodes a loop structure where the number of repetitions (Loop-Bound) is determined dynamically based on the request’s actual token length, not a pre-defined maximum token length.
- **Dyn-Modi** instruction operates within this loop to adjust target operand fields (e.g., the row/col address for a MAC instruction) using a specified stride. This mechanism effectively generates a virtual address that is translated to a physical address on-the-fly, enabling the PIM to access dynamically allocated and non-contiguous KV cache memory—a capability impossible for conventional PIM controllers.

This dynamic and compact representation provides an advantage over prior PIM systems ([16], [21], [54], [62]), which must generate instruction sequences whose size grows linearly with the token length. As shown in Fig. 10(c), this linear growth creates severe instruction buffer pressure and a scalability bottleneck for long-context inference. In comparison, DPA’s encoding ensures the instruction size remains small and nearly constant regardless of context length, thereby avoiding command buffer bloat and enabling scalability to long context.

C. On-Module PIM Instruction Dispatcher

To support DPA, we enhance the PIM HUB (Fig.3(c)) with a lightweight on-module dispatcher that handles dynamic instruction generation and address translation. As shown in Fig. 11(a), this dispatcher consists of an instruction buffer to hold the compact, DPA-encoded instruction sequence; a configuration buffer for per-request information like request ID and token index (T_{cur}); and a Virtual-to-Physical Address (VA2PA) translation table to maintain the mapping between logical virtual addresses and the physical memory locations. The physical memory space is allocated in a granularity of *chunk*, defined as $channels \times banks \times rows$. A decoding unit combines the DPA-encoded instructions, the per-request information, and the resolved physical addresses from the VA2PA table to generate the final executable PIM instruction sequence. After decoding, the instructions are staged for execution in the Instruction Queue inside the PIM HUB’s Instruction Sequencer (Fig. 3(a)).

Before LLM inference starts, the host initializes the dispatcher with per-request metadata, including the VA2PA table

and the initial token length. Once assigned, the dispatcher manages token progression by incrementing the token length after each generation step, maintaining this state until the request completes. During this process, no host intervention is required. Host–PIM communication occurs only when a new request is assigned to a module, when an active request requires additional memory space, or when a completed request is released. As this communication does not occur on every decoding step, its overhead is negligible in practice. This localized update mechanism allows the dispatcher to efficiently handle variable-length decoding while minimizing communication latency between the host and PIM modules.

During instruction decoding, the Decoder reads operand addresses from the instruction buffer and resolves them through the VA2PA table according to the active request context. For example, as shown in Fig. 11(a), a MAC instruction with a virtual address of 0 can be dynamically translated to different physical addresses depending on the request ID—it resolves to address 22 for Request 1 but maps to address 33 for Request 2. This dynamic address translation enables the dispatcher to support non-contiguous and dynamically allocated memory regions across concurrent requests. Importantly, instruction decoding is performed in a pipelined manner with instruction execution. Decoded instructions are staged into the instruction queue while previously decoded instructions are executed, allowing decoding and execution to proceed in parallel without introducing additional stalls on the critical path.

Lazy Memory Allocation. The integration of DPA and the on-module dispatcher enables efficient *lazy memory allocation*, directly addressing memory underutilization in static PIM systems. Instead of reserving memory for worst-case scenarios, the host allocates 1MB chunks on demand as a request’s KV cache grows, updating the VA2PA table accordingly. As shown in Fig.11(b), when more memory is needed, a new chunk is allocated and mapped non-contiguously via virtual addresses. This dynamic scheme reduces internal fragmentation to only the last-used chunk per request. Compared to static T_{max} reservation, it significantly improves memory capacity efficiency (as discussed later in Sec.VIII-C).

VII. PIMPHONY IMPLEMENTATION

PIMphony integrates TCP, DCS, and DPA through compiler and runtime techniques. To demonstrate its benefit, we apply our MLIR-based implementation to existing simulators for NeuPIMs [21] (NPU+PIM) and CENT [16] (PIM-only), enabling evaluation across different types of PIM systems.

A. Multi-PIM Compiler and Runtime

We implement PIMphony on top of MLIR [64], a flexible compiler framework designed for modular optimization across software and hardware layers (Fig. 12). By extending MLIR’s dialects, we generate PIM-specific code for Attention and feed-forward subgraphs in Transformer decoding workloads. PIMphony’s custom pattern-matching and code generation passes target PIM-amenable kernels (e.g., QK^T , SV , FFN),

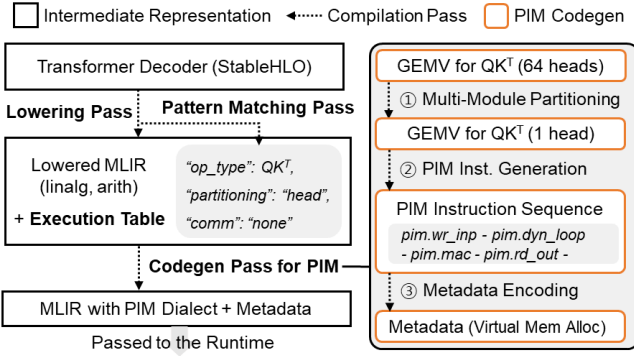


Fig. 12: Overall compilation flow of PIMphony.

embedding dynamic partitioning and memory allocation metadata. Compilation is performed offline and does not affect inference latency.

For runtime support, we enhance the IREE [67] runtime stack and its hardware abstraction layer (HAL) to interface with PIM SDKs provided by commercial platforms [62]. This enables the deployment of PIMphony’s instruction sequences in realistic multi-node settings, where token-centric partitioning and dynamic memory management adaptively respond to variations of context length. These runtime extensions are implemented without assuming new hardware primitives, ensuring compatibility with NeuPIMs and CENT simulators.

B. PIM System Integration

To evaluate PIMphony in multi-node inference settings, we incorporate the compiler-generated PIM instruction sequences into simulators for both NeuPIMs and CENT. In NeuPIMs, PIMphony offloads bandwidth-bound Attention operations to PIM modules, while in CENT, all computations are handled with PIM. We apply PIMphony’s hardware-aware optimizations—including on-module dispatch logic and I/O buffering—within the simulation backends of both platforms, with DRAM command timing and resource contention calibrated with AiMX [62] PIM specifications.

C. Hardware Overhead

PIMphony’s proposed hardware modifications are lightweight, as the I/O-aware buffering incurs a minimal area overhead, occupying just 0.47% of the MAC unit area [40] per PIM bank, estimated by synthesizing the output buffer logic using CACTI [8]. In addition, enabling DCS within PIM controllers in PIM HUB [30], [35], [37] adds only a negligible controller cost: across all HUB control blocks [62], the area increases by 0.5% and the power by 1.3%. This overhead accounts for the additional dependency-tracking structures and logic required by DCS, including a per-controller 576B metadata table (D-Table and S-Table) and the associated dependency-check unit used to validate per-entry hazards. The on-module dispatcher supporting DPA incurs a 4% area overhead. This dispatcher requires less than 200KB for all its internal buffers (VA2PA, command, configuration), which is much smaller than the 512KB GPR capacity in a typical PIM

TABLE IV: PIMphony module configurations.

NeuPIMs [21]	Compute	8 Matrix Units(256 TFLOPS), 32 PIM channels
	Memory	32GB
	Internal BW	32TB/s
CENT [16]	Compute	PNM (3 TFLOPS), 32 PIM channels
	Memory	16GB
	Internal BW	16TB/s

HUB, ensuring efficient integration without significant area pressure.

VIII. EVALUATION

A. Evaluation Settings

We evaluate PIMphony using a range of LLMs (Table I) with context lengths up to 128K, across four tasks from LongBench [7] and LV-Eval [73] (Table II). Non-GQA models (LLM-7B/72B-32K) are tested with LongBench, while GQA-enabled models (LLM-7B/72B-128K) are evaluated with LV-Eval. For comparison, we use two PIM-based baselines: CENT [16], a PIM-only system with 16GB per module, and NeuPIMs [21], a hybrid xPU+PIM system with 32GB per module. We modify both simulators [16], [21] to integrate our techniques, configuring 128GB for 7B and 512GB for 72B models, following prior PIM studies. PIMphony is modeled using a validated Ramulator-based simulator incorporating AiMX [30], [35], [37] architecture, and is evaluated using the parameters detailed in Table IV.

B. Performance Evaluation.

We evaluate the end-to-end throughput improvements of our proposed techniques across both PIM-only (CENT [16]) and heterogeneous xPU+PIM systems (NeuPIMs [21]). As shown in Fig. 13 and 14, we incrementally apply three optimizations of PIMphony: token-centric PIM partitioning (TCP), dynamic PIM command scheduling (DCS), and dynamic PIM access (DPA).

Overall Throughput Gains. Performance consistently improves with the progressive application of the proposed methods. In Fig. 13(a) and 14(a), we observe a 2.1-4.5x speedup for GQA-disabled models (LLM-7B-32K, LLM-72B-32K) evaluated on LongBench. For GQA-enabled models (LLM-7B-128K, LLM-72B-128K) evaluated on LV-Eval (up to 128K tokens), our methods achieve up to 11.3x speedup, demonstrating their strong effectiveness in large-context scenarios. Notably, PIMphony’s relative throughput gain is often higher for the 72B models because larger models suffer more from baseline inefficiencies (e.g., PIM underutilization in the CENT system), creating greater room for improvement. While this trend is evident in PIM-only systems, the advantage in xPU+PIM systems becomes more pronounced under very long-context workloads (e.g., 128K), where PIM-side execution increasingly dominates.

Effectiveness of TCP, DCS, and DPA. TCP directly addresses the poor channel utilization in PIM-only and xPU+PIM systems, which is caused by the small batch sizes and context imbalances inherent to long-context workloads, more pronounced in longer context (LV-Eval). While TCP effectively

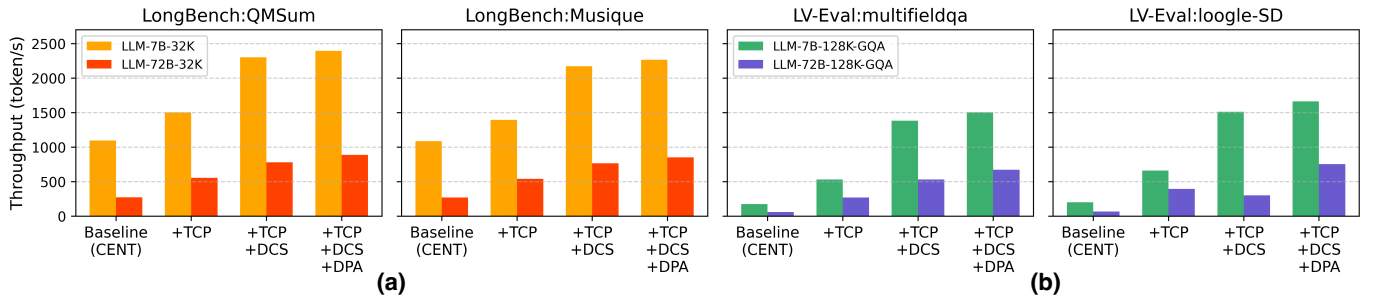


Fig. 13: Throughput of PIM-only systems — 7B: 8 modules (128GB), 70B: 32 modules (512GB). (a) Non-GQA LLMs on LongBench. (b) GQA-enabled LLMs on LV-Eval. Each bar shows improvements from TCP, DCS, and DPA, using optimal TP/PP settings.

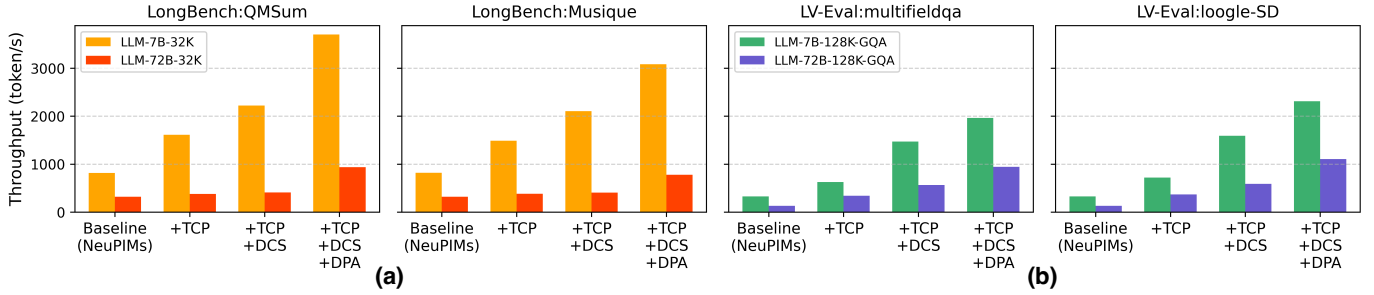


Fig. 14: Throughput of xPU-PIM systems — 7B: 4 modules (128GB), 70B: 16 modules (512GB). (a) Non-GQA LLMs on LongBench. (b) GQA-enabled LLMs on LV-Eval. Each bar shows improvements from TCP, DCS, and DPA, using optimal TP/PP settings.

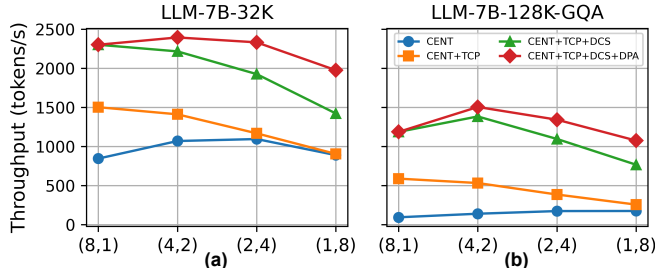


Fig. 15: Throughput of various (TP,PP) on (a) LLM-7B-32K with LongBench QMSum (b) LLM-7B-128K-GQA with multifieldqa.

boosts PIM utilization, the system can become I/O-bound, a problem that DCS resolves by overlapping computation and data movement. This synergy is especially impactful for GQA-enabled models, where DCS unlocks the gains of *row-reuse mapping* by overlapping the additional I/O transfers (especially WR-INP) with MAC execution, thereby amplifying ACT/PRE savings over TCP-only configurations. Finally, in heterogeneous xPU+PIM systems, DPA plays a critical role in maintaining system-level balance. As TCP and DCS accelerate the PIM-bound Attention stage, the xPU-bound FC stage increasingly becomes the dominant performance bottleneck. DPA alleviates this imbalance by enabling larger batch sizes, which in turn improves NPU utilization and maximizes end-to-end throughput.

C. Ablation Study

In this section, we evaluate the effectiveness of PIMphony across several key metrics: multi-PIM parallelization, latency and energy breakdowns, system and context length scalability, memory capacity utilization, and a direct throughput

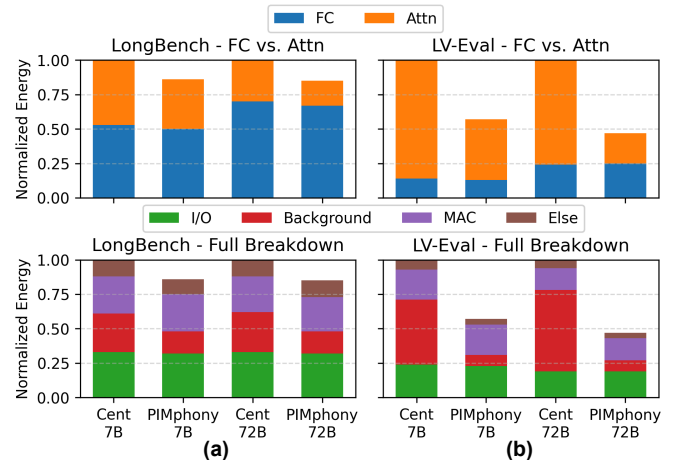


Fig. 16: Energy breakdowns of CENT vs. CENT+PIMphony for 7B and 72B models: (a) non-GQA models evaluated on LongBench (32K), (b) GQA models evaluated on LV-Eval (128K). Top: FC and Attention. Bottom: MAC, I/O, Background, and Else.

comparison with GPU systems. Unless otherwise noted, we present results primarily from the PIM-only system to more distinctively illustrate the impact of PIMphony on PIM itself.

Tensor vs. Pipeline Parallelization. Fig. 15 illustrates the impact of PIMphony’s techniques on multi-node PIM parallelization, evaluated on CENT. We incrementally apply TCP, DCS, and DPA to assess their cumulative benefits. For LLM-7B-32K (woGQA), TCP significantly enhances TP efficiency by mitigating channel underutilization. DCS further improves Attention performance by reducing I/O idle time, while DPA increases the effective batch size, enabling PP to deliver marginally higher throughput. For LLM-7B-128K (wGQA),

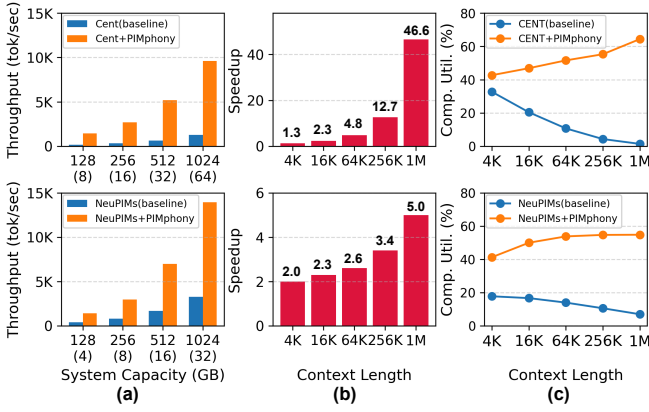


Fig. 17: Scalability of PIMphony on LLM-7B-128K-wGQA with 3-sigma context variation for CENT (Top) and NeuPIMs (Bottom). (a) At 64K context, throughput scales with capacity (128–1024GB; 8–64 modules for CENT, 4–32 for NeuPIMs). (b)(c) At 512GB, PIMphony outperforms baselines as context length increases from 4K to 1M.

DCS shows a more pronounced effect due to the intensified input transfer pressure from GQA. Additionally, KV cache reuse from GQA, combined with DPA, boosts batch size and further motivates PP, achieving 20% higher throughput improvement. These results demonstrate the effectiveness of PIMphony in maximizing multi-PIM system efficiency.

Energy Breakdown. PIMphony’s energy efficiency stems from its dramatic reduction of runtime-dependent background energy. As shown in Fig. 16, the baseline’s low MAC utilization causes background energy to constitute a staggering 71.5% of the total energy in Attention layers. By accelerating Attention execution by up to $19\times$, PIMphony drastically cuts the runtime, which in turn slashes the background energy’s share to just 13.0% and achieves up to a $3.46\times$ reduction in Attention. These gains are amplified in GQA-enabled models, as their higher speedup potential leads to an even greater reduction in runtime and, consequently, background energy.

Scalability with System Capacity and Context Length. PIMphony scales robustly with both system capacity and context length, widening its lead over baselines. Throughput improves with capacity (128GB–1024GB), confirming efficient module use (Fig.17(a)). Benefits are greater when scaling context length to 1M tokens on a fixed 512GB system. CENT collapses under pipeline bubbles, dropping to 2% utilization, while PIMphony achieves $46.6\times$ speedup. NeuPIMs scale more stably via tensor parallelism, yet PIMphony still delivers $5.0\times$ speedup (Fig.17(b)). Fig. 17(c) explains this trend: PIMphony makes Attention more efficient than FC layers, so longer contexts—where Attention dominates—boost system utilization, unlike baselines where bottlenecks worsen. Importantly, these gains are not confined to long contexts; even at short contexts (e.g., 256 tokens), PIMphony achieved a $2.1\times$ speedup over the baselines.

DCS vs. Ping-pong Buffering Prior works [25], [29] adopt *ping-pong buffering*: a single buffer is split into two regions so two operations that touch disjoint regions can overlap—thus, like DCS, it can overlap I/O transfers (WR-INP, RD-OUT)

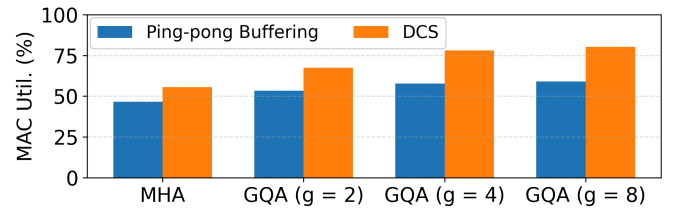


Fig. 18: Compute utilization for Attention operations comparing *ping-pong buffering* and DCS. X-axis: MHA and GQA with group size $g \in \{2, 4, 8\}$. Both apply the *row-reuse mapping* in GQA.

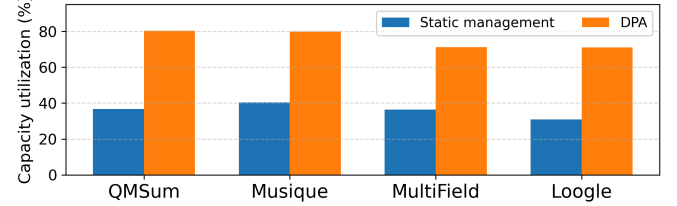


Fig. 19: Capacity utilization with and without DPA in PIMphony. QMSum/Musique use 7B-32K; MultiField/Loogle use 7B-128K.

with MAC to improve MAC utilization. However, because *static scheduling* does not know true data dependencies at the entry level, I/O transfers and MAC cannot access the same region concurrently to avoid data hazards; effectively, overlap is restricted to different regions. Consequently, when I/O transfers and MAC must switch which region they operate on, the swap can occur only after both regions become idle, which introduces hand-off pipeline stalls. In contrast, DCS keeps one buffer, tracks per-entry dependencies, and relaxes inter-command timing to enable entry-level overlap within the same buffer without hand-offs, yielding a longer, stall-resistant pipeline. With the same total buffer size, our ping-pong baseline for I/O hiding exhibits hand-off pipeline stalls—exacerbated by shorter pipelines from smaller per-region buffers—whereas DCS sustains overlap; across Attention settings, DCS achieves up to $1.4\times$ higher compute-unit utilization (Fig. 18).

Impact of DPA on Capacity Utilization. To assess DPA’s capacity efficiency, we conduct an ablation comparing static memory management with DPA’s dynamic approach. As shown in Fig. 19, static memory management severely underutilizes memory—capacity utilization ranges from 31.0% to 40.5% across workloads—due to coarse-grained reservations sized for maximum token lengths, which over-provision capacity. DPA instead uses lazy, on-demand allocation of non-contiguous chunks at runtime. Although this introduces minor fragmentation limited to each request’s final chunk, it removes maximum-length reservations and markedly improves efficiency. Empirically, DPA raises average capacity utilization to 75.6%, more than doubling the static baseline.

Throughput Comparison with GPU System. We compare PIMphony-enabled systems with a strong GPU baseline—A100s with flash-decoding (FD) [11] and paged-attention (PA) [34]—using memory-matched configurations (Fig. 20). PIMphony delivers substantial speedups, especially

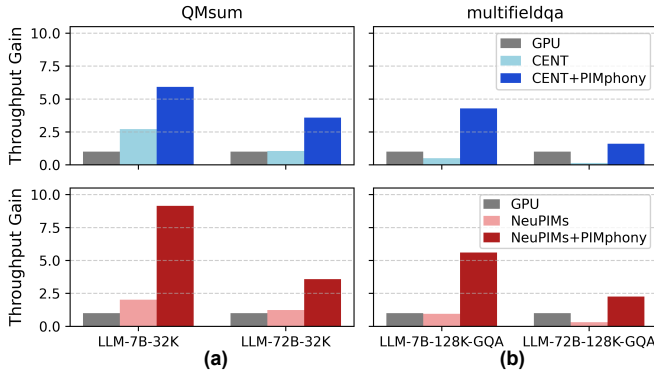


Fig. 20: Throughput comparison of GPU and PIMphony. (a) Non-GQA LLM on QMSum (LongBench). (b) GQA-enabled LLM on multifieldqa (LV-Eval). For fairness, GPU memory is matched to PIMphony: two A100-80GB for LLM-7B, eight for LLM-72B.

on non-GQA models with high memory demand. On larger 72B models, the GPU’s advantage on compute-intensive FC layers narrows PIMphony’s relative gains. Even so, PIMphony shows a key strength on GQA workloads: while KV cache reuse benefits GPUs, it creates an I/O bottleneck for PIMs due to repeated input transfers. Our DCS technique mitigates this by overlapping transfers with computation, hiding the overhead and unlocking significant gains even in workloads traditionally favorable to GPUs.

IX. RELATED WORK

Multi-Node LLM Serving. As LLMs scale, their memory and bandwidth demands grow, making parallelization—such as tensor [61] and pipeline parallelism [23]—crucial for serving. Modern systems [1]–[3] further improve GPU utilization with continuous batching [72] and efficient KV cache management [34]. Still, GPU underutilization persists during token generation [18], [55]. This paper targets improved PIM utilization for long-context LLM inference, where GPUs face bandwidth and capacity bottlenecks. We propose architectural and runtime techniques—token-centric partitioning, dynamic command scheduling, and dynamic memory management—that boost MAC and memory efficiency in both PIM-only and xPU+PIM systems.

DRAM-Based PIM. DRAM-based Processing-in-Memory (PIM) is a key solution for accelerating LLMs, with general-purpose platforms like UPMEM-PIM [14], [17], [24] and domain-specific accelerators such as HBM-PIM [28], [31], [32], [38], [41] and AiM [19], [33], [40]. These are optimized for tasks like matrix multiplication. Recent work addresses limitations in both homogeneous [27], [75] and heterogeneous xPU–PIM systems [21], [44], [54], [59], [69]. AiMX [30], [35], [37], [62], which integrates FPGA with AiM, further advances compute and parallelism for LLMs, signaling a shift toward specialized, efficient hardware.

DRAM-Based PIM for LLM Inference. DRAM-based PIM architectures are gaining attention for accelerating memory-bound phases of LLM inference by leveraging high internal bandwidth near memory arrays. Recent work explores

compiler co-optimization, dynamic scheduling, and CXL-based system integration to support complex model execution on PIM [12], [20], [43], [63], highlighting a shift toward holistic hardware–software co-design for LLM acceleration.

PIM-Based Memory Management Units. Only limited prior work [39] has examined memory management support in Processing-in-Memory architectures. The most relevant effort is the MMU design proposed for the UPMEM system [14], which provides an address translation facility primarily to manage data transfers between DRAM and each PIM core’s local SRAM. This mechanism operates similarly to a DMA engine and is optimized for orchestrating bulk data movement across the DRAM–SRAM boundary. However, its scope is restricted to fixed data-transfer pipelines and does not address the challenges associated with dynamic memory allocation or operand remapping within DRAM itself. In contrast to these prior approaches, our work performs virtual-to-physical address translation directly inside the PIM. This design enables on-demand address resolution and dynamic KV cache allocation within DRAM-based PIM modules, thereby supporting the flexible memory management required by modern LLM inference systems.

Digital Accelerators for LLM Inference Recent digital accelerator studies [22], [46], [49], [56], [58] have explored various architectural optimizations to enhance transformer-based LLM inference efficiency. Among them, Hybe [49] directly addresses the long-context LLM inference caused by the imbalance between compute-intensive prefill and memory-intensive decode stages. It adopts a GPU–NPU hybrid architecture, using GPUs for the prefill stage and lightweight NPUs for the decode stage to maximize hardware utilization. In addition, Hybe employs fine-grained KV transmission and stage-wise pipelining to reduce GPU memory pressure and minimize hardware stalls.

X. CONCLUSIONS

We propose PIMphony, a multi-node PIM acceleration system featuring an MLIR-based compiler and novel techniques for partitioning, scheduling, and memory management. By systematically resolving key bottlenecks, PIMphony significantly enhances throughput for long-context LLM inference up to $11.3\times$ and $8.4\times$ on PIM-only and xPU+PIM systems, respectively.

ACKNOWLEDGMENT

This work was partly supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2023-00260527 and No. RS-2025-00561961). This work was also partly supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2022-0-00971, Logic Synthesis for NVM-based PIM Computing Architecture, and No. RS-2020-II201373, Artificial Intelligence Graduate School Program (Hanyang University)). This work was also partly supported by SK Hynix Inc.

REFERENCES

- [1] "NVIDIA TensorRT-LLM." [Online]. Available: <https://github.com/NVIDIA/TensorRT-LLM>
- [2] "SGLang." [Online]. Available: <https://github.com/sgl-project/sglang>
- [3] "vLLM." [Online]. Available: <https://github.com/vllm-project/vllm>
- [4] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," 2023. [Online]. Available: <https://arxiv.org/abs/2305.13245>
- [5] K. Asifuzzaman, N. R. Miniskar, A. R. Young, F. Liu, and J. S. Vetter, "A survey on processing-in-memory techniques: Advances and challenges," *Memories - Materials, Devices, Circuits and Systems*, vol. 4, p. 100022, 2023.
- [6] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, B. Hui, L. Ji, M. Li, J. Lin, R. Lin, D. Liu, G. Liu, C. Lu, K. Lu, J. Ma, R. Men, X. Ren, X. Ren, C. Tan, S. Tan, J. Tu, P. Wang, S. Wang, W. Wang, S. Wu, B. Xu, J. Xu, A. Yang, H. Yang, J. Yang, S. Yang, Y. Yao, B. Yu, H. Yuan, Z. Yuan, J. Zhang, X. Zhang, Y. Zhang, Z. Zhang, C. Zhou, J. Zhou, X. Zhou, and T. Zhu, "Qwen technical report," 2023. [Online]. Available: <https://arxiv.org/abs/2309.16609>
- [7] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "Longbench: A bilingual, multitask benchmark for long context understanding," 2024. [Online]. Available: <https://arxiv.org/abs/2308.14508>
- [8] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "Cacti 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Trans. Archit. Code Optim.*, vol. 14, no. 2, jun 2017. [Online]. Available: <https://doi.org/10.1145/3085572>
- [9] Claude, "Claude 3.5 sonnet," <https://www.anthropic.com/news/claude-3-5-sonnet>, 2024, accessed on: 2024-11-21.
- [10] G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen, L. Marris, S. Petulla, C. Gaffney, A. Aharoni, N. Lintz, T. C. Pais, H. Jacobsson, I. Szpektor, N.-J. Jiang, K. Haridasan, A. Omran, N. Saunshi, D. Bahri, G. Mishra, E. Chu, T. Boyd, B. Hekman, A. Parisi, C. Zhang, K. Kawintiranon, T. Bedrax-Weiss, O. Wang, Y. Xu, O. Purkiss, U. Mendlovic, I. Deutel, N. Nguyen, A. Langley, F. Korn, L. Rossazza, A. Ramé, S. Waghmare, H. Miller, N. Byrd, A. Sheshan, R. Hadsell, S. Bhardwaj, P. Janus, T. Rissa, D. Horgan, A. Abdagic, L. Belenki, J. Allingham, A. Singh, T. Guidroz, S. Srinivasan, H. Schmit, K. Chiafullo, A. Elisseeff, N. Jha, P. Kolhar, L. Berrada, F. Ding, X. Si, S. B. Mallick, F. Och, S. Erell, E. Ni, T. Latkar, S. Yang, P. Sirkovic, Z. Feng, R. Leland, R. Hornung, G. Wu, C. Blundell, H. Alvari, P.-S. Huang, C. Yip, S. Deur, L. Liu, G. Surita, P. Duque, D. Damen, J. Jia, A. Guez, M. Mircea, A. Sinha, A. Magni, P. Stradomski, T. Marian, V. Galić, W. Chen, H. Husain, A. Singhal, D. Grewe, F.-X. Aubet, S. Song, L. Blanco, L. Rechis, L. Ho, R. Munoz, K. Zheng, J. Hamrick, K. Mather, H. Taitelbaum, E. Rutherford, Y. Lei, K. Chen, A. Shukla, E. Moreira, E. Doi, B. Isik, N. Shabat, D. Rogozińska, K. Kolipaka, J. Chang, E. Vušak, S. Venkatachary, S. Noghabi, T. Bharti, Y. Jun, A. Zaks, S. Green, J. Challagundla, W. Wong, M. Mohammad, D. Hirsch, Y. Cheng, I. Naim, L. Proleev, D. Vincent, A. Singh, M. Krikun, D. Krishnan, Z. Ghahramani, A. Atias, R. Aggarwal, C. Kirov, D. Vytiniotis, C. Koh, A. Chronopoulou, P. Dogra, V.-D. Ion, G. Tyen, J. Lee, F. Weissenberger, T. Strohm, A. Balakrishna, J. Rae, M. Velic, R. de Liedekerke, O. Elyada, W. Yuan, C. Liu, L. Shani, S. Kishchenko, B. Alessio, Y. Li, R. Song, S. Kwei, O. Jankowski, A. Pappu, Y. Namiki, Y. Ma, N. Tripuraneni, C. Cherry, M. Ikonomidis, Y.-C. Ling, C. Ji, B. Westberg, A. Wright, D. Yu, D. Parkinson, S. Ramaswamy, J. Connor, S. H. Yeganeh, S. Grover, G. Kenwright, L. Litchev, C. Apps, A. Tomala, F. Halim, A. Castro-Ros, Z. Li, A. Boral, P. Sho, M. Yarom, E. Malmi, D. Klinghoffer, R. Lin, A. Ansell, P. K. S. Zhao, S. Zuo, A. Santoro, H.-T. Cheng, S. Demmessie, Y. Liu, N. Brichtova, A. Culp, N. Braun, D. Graur, W. Ng, N. Mehta, A. Phillips, P. Sundberg, V. Godbole, F. Liu, Y. Katariya, D. Rim, M. Seyedhosseini, S. Ammirati, J. Valfridsson, M. Malihi, T. Knight, A. Toor, T. Lampe, A. Ittycheriah, L. Chiang, C. Yeung, A. Fréchette, J. Rao, H. Wang, H. Srivastava, R. Zhang, R. Rhodes, A. Brand, D. Weesner, I. Figotin, F. Gimeno, R. Fellingner, P. Marcenac, J. Leal, E. Marcus, V. Cotruta, R. Cabrera, S. Luo, D. Garrette, V. Axelrod, S. Baltateanu, D. Barker, D. Chen, H. Toma, B. Ingram, J. Riesa, C. Kulkarni, Y. Zhang, H. Liu, C. Wang, M. Polacek, W. Wu, K. Hui, A. N. Reyes, Y. Su, M. Barnes, I. Malhi, A. Siddiqui, Q. Feng, M. Damaschkin, D. Pighin, A. Steiner, S. Yang, R. S. Boppana, S. Ivanov, A. Kandoor, A. Shah, A. Mujika, D. Huang, C. A. Choquette-Choo, M. Patel, T. Yu, T. Creswell, Jerry, Liu, C. Barros, Y. Razeghi, A. Roy, P. Culliton, B. Xiong, J. Pan, T. Strohm, T. Powell, B. Seal, D. DeCarlo, P. Shyam, K. Katircioglu, X. Wang, C. Hardin, I. Odisho, J. Broder, O. Chang, A. Nair, A. Shtefan, M. O'Brien, M. Agarwal, S. Potluri, S. Goyal, A. Jhinal, S. Thakur, Y. Stuken, J. Lyon, K. Toutanova, F. Feng, A. Wu, B. Horn, A. Wang, A. Cullum, G. Taubman, D. Shrivastava, C. Shi, H. Tomlinson, R. Patel, T. Tu, A. M. Oflazer, F. Pongetti, M. Yang, A. A. Taiga, V. Perot, N. W. Pierse, F. Han, Y. Drori, I. Iturrate, A. Chakrabarti, L. Yeung, D. Dopson, Y. ting Chen, A. Kulshreshtha, T. Guo, P. Pham, T. Schuster, J. Chen, A. Polozov, J. Xing, H. Zhou, P. Kacham, D. Kuklianskiy, A. Miech, S. Yaroshenko, E. Chi, S. Douglas, H. Fei, M. Blondel, P. Myla, L. Madmoni, X. Wu, D. Keysers, K. Kjems, I. Albuquerque, L. Yu, J. D'sa, M. Plantan, V. Ionescu, J. S. Elias, A. Gupta, M. R. Vuyyuru, F. Alcober, T. Zhou, K. Ji, F. Hartmann, S. Puttagunta, H. Song, E. Amid, A. Stefanoiu, A. Lee, P. Pucciarelli, E. Wang, A. Raul, S. Petrov, I. Tian, V. Ankin, N. Nti, V. Gomes, M. Schumacher, G. Vesom, A. Panagopoulos, K. Bousmalis, D. Andor, J. Jacob, Y. Zhang, B. Rosgen, M. Kecman, M. Tung, A. Belias, N. Goodman, P. Covington, B. Wieder, N. Saxena, E. Davoodi, M. Huang, S. Maddineni, V. Roulet, F. Campbell-Ajala, P. G. Sessa, Xintian, Wu, G. Lai, P. Collins, A. Haig, V. Sakenas, X. Xu, M. Giustina, L. E. Shafey, P. Charoenpanit, S. Garg, J. Ainslie, B. Severson, M. G. Arenas, S. Pathak, S. Rajayogam, J. Feng, M. Bakker, S. Li, N. Wichers, J. Rogers, X. Geng, Y. Li, R. Jagerman, C. Jia, N. Olmert, D. Sharon, M. Mauger, S. Mariserla, H. Ma, M. Mohabey, K. Kim, A. Andreev, S. Pollom, J. Love, V. Jain, P. Agrawal, Y. Schroecker, A. Fortin, M. Warmuth, J. Liu, A. Leach, I. Blok, G. P. Girirajan, R. Aharoni, B. Uria, A. Sozanschi, D. Goldberg, L. Ionia, M. T. Ribeiro, M. Zlocha, V. Birodkar, S. Lachgar, L. Yuan, H. Choudhury, M. Ginsberg, F. Zheng, G. Dibb, E. Graves, S. Lokhande, G. Rasskin, G.-C. Muraru, C. Quick, S. Tata, P. Sermanet, A. Chawla, I. Karo, Y. Wang, S. Zhang, O. Keller, A. Dragan, G. Su, I. Chou, X. Liu, Y. Tao, S. Prabhakara, M. Wilson, R. Liu, S. Wang, G. Evans, D. Du, A. Castaño, G. Prasad, M. E. Mahdy, S. Gerlach, M. Reid, J. Kahn, A. Zait, T. S. Pillai, T. Ulrich, G. Wang, J. Wassenberg, E. Farkash, K. Yalasangi, C. Wang, M. Bauza, S. Bucher, T. Liu, J. Yan, G. Leung, V. Sindhwani, P. Barnes, A. Singh, I. Jurin, J. Chang, N. K. Bhumiher, S. Eiger, G. Citovsky, B. Withbroe, Z. Li, S. Xue, N. D. Santo, G. Stoyanov, Y. Raimond, S. Zheng, Y. Gao, V. Listfk, S. Kwasiorsk, R. Saputro, A. Ozturk, G. Mallya, K. Majmundar, R. West, P. Caron, J. Wei, L. Castrejon, S. Vikram, D. Ramachandran, N. Dhawan, J. Park, S. Smoot, G. van den Driessche, Y. Blau, C. Malik, W. Liang, R. Hirsch, C. N. dos Santos, E. Weinstein, A. van den Oord, S. Lall, N. FitzGerald, Z. Jiang, X. Yang, D. Webster, A. Elqursh, A. Pope, G. Rotival, D. Raposo, W. Zhu, J. Dean, S. Alabed, D. Tran, A. Gupta, Z. Gleicher, J. Austin, E. Rosseel, M. Umekar, D. Das, Y. Sun, K. Chen, K. Misiunas, X. Zhou, Y. Di, A. Loo, J. Newlan, B. Li, V. Ramasesh, Y. Xu, A. Chen, S. Gandhe, R. Soricut, N. Gupta, S. Hu, S. El-Sayed, X. Garcia, I. Brusilovsky, P.-C. Chen, A. Bolt, L. Huang, A. Gurney, Z. Zhang, A. Pritzel, J. Willkiewicz, B. Seybold, B. K. Shamanna, F. Fischer, J. Dean, G. Gill, R. McIlroy, A. Bhowmick, J. Selier, A. Yang, D. Cheng, V. Magay, J. Tan, D. Varma, C. Walder, T. Kocisky, R. Nakashima, P. Natsev, M. Kwong, I. Gog, C. Zhang, S. Dieleman, T. Jimma, A. Ryabtsev, S. Brahma, D. Steiner, D. Du, A. Žužul, M. Žanić, M. Raghavachari, W. Gierke, Z. Zheng, D. Petrova, Y. Dauphin, Y. Liu, I. Kessler, S. Hand, C. Duvarney, S. Kim, H. Lee, L. Hussenot, J. Hui, J. Smith, D. Jain, J. Xia, G. S. Tomar, K. Amiri, D. Phan, F. Fuchs, T. Weyand, N. Tomasev, A. Cordell, X. Liu, J. Mallinson, P. Joshi, A. Crawford, A. Suggala, S. Chien, N. Fernando, M. Sanchez-Vargas, D. Williams, P. Crone, X. Luo, I. Karpov, J. Shan, T. Thürk, R. Strudel, P. Voigtlaender, P. Patil, T. Dozat, A. Khodaei, S. Singla, P. Ambroszczyk, Q. Wu, Y. Chang, B. Roark, C. Hegde, T. Ding, A. Filos, Z. Wu, A. S. Pinto, S. Liu, S. Khanna, A. Pandey, S. Mccloughlin, Q. Li, S. Haves, A. Zhou, E. Buchatskaya, I. Leal, P. de Boursac, N. Akazawa, N. Anderson, T. Chen, K. Somandepalli, C. Liang, S. Goenka, S. Winkler, A. Grushetsky, Y. Ding, J. Smith, F. Ye, J. Pont-Tuset, E. Li, R. Li, T. Golany, D. Wegner, T. Jiang, O. Barak, Y. Shanguan, E. Vértés, R. Wong, J. Bornschein, A. Tudor, M. Bevilacqua, T. Schaul, A. S. Rawat, Y. Zhao, K. Axiotis,

- L. Meng, C. McLean, J. Lai, J. Beattie, N. Kushman, Y. Liu, B. Kutzman, F. Lang, J. Ye, P. Netrapalli, P. Mishra, M. Khan, M. Goel, R. Willoughby, D. Tian, H. Zhuang, J. Chen, Z. Tsai, T. Kementsietsidis, A. Khare, J. Keeling, K. Xu, N. Waters, F. Althé, A. Popat, B. Mittal, D. Saxton, D. E. Badawy, M. Mathieu, Z. Zheng, H. Zhou, N. Ranka, R. Shin, Q. Duan, T. Salimans, I. Mihailescu, U. Shaham, M.-W. Chang, Y. Assael, N. Dikkala, M. Izzard, V. Cohen-Addad, C. Graves, F. Feinberg, G. Chung, D. Strouse, D. Karmon, S. Sharifzadeh, Z. Ashwood, K. Pham, J. Blanton, A. Vasiloff, J. Barber, M. Geller, A. Zhou, F. Zubach, T.-K. Huang, L. Zhang, H. Gupta, M. Young, J. Proskurnia, R. Votel, V. Gabeur, G. Barcik, A. Tripathi, H. Yu, G. Yan, B. Changpinyo, F. Pavetić, A. Coyle, Y. Fujii, J. G. Mendez, T. Zhou, H. Rajamani, B. Hechtman, E. Cao, D.-C. Juan, Y.-X. Tan, V. Dalibard, Y. Du, N. Clay, K. Yao, W. Jia, D. Vijaykumar, Y. Zhou, X. Bai, W.-C. Hung, S. Pecht, G. Todorov, N. Khadke, P. Gupta, P. Lahoti, A. Atef, K. Duddu, J. Lee-Thorp, A. Bykovsky, T. Misiunas, S. Flennerhag, S. Thangaraj, J. McGiffin, Z. Nado, M. Kunesch, A. Noever, A. Hertz, M. Liang, V. Stone, E. Palmer, S. Daruki, A. Pramanik, S. Pöder, A. Kyker, M. Khan, E. Sluzhaev, M. Ritter, A. Ruderman, W. Zhou, C. Nagpal, K. Vodrahalli, G. Necula, P. Barham, E. Pavlick, J. Hartford, I. Shaffran, L. Zhao, M. Mikula, T. Eccles, H. Shimokawa, K. Garg, L. Vilnis, H. Chen, I. Shumailov, K.-H. Lee, A. Abdelhamed, M. Xie, V. Cohen, E. Hlavnova, D. Malkin, C. Sitawarin, J. Lottes, P. Coquino, T. Yu, S. Kumar, J. Zhang, A. Mahendru, Z. Ahmed, J. Martens, T. Chen, A. Boag, D. Peng, C. Devin, A. Klimovskiy, M. Phuong, D. Vainstein, J. Xie, B. Ramabhadran, N. Howard, X. Yu, G. Goswami, J. Cui, S. Shleifer, M. Pinto, C.-K. Yeh, M.-H. Yang, S. Javanmardi, D. Ethier, C. Lee, J. Orbay, S. Kotecha, C. Bromberg, P. Shaw, J. Thornton, A. G. Rosenthal, S. Gu, M. Thomas, I. Gemp, A. Ayyar, A. Ushio, A. Selvan, J. Wee, C. Liu, M. Majzoubi, W. Yu, J. Abernethy, T. Liechty, R. Pan, H. Nguyen, Qiong, Hu, S. Perrin, A. Arora, E. Pitler, W. Wang, K. Shivakumar, F. Prost, B. Limonchik, J. Wang, Y. Gao, T. Cour, S. Buch, H. Gui, M. Ivanova, P. Neubeck, K. Chan, L. Kim, H. Chen, N. Goyal, D.-W. Chung, L. Liu, Y. Su, A. Petrushkina, J. Shen, A. Joulain, Y. Xu, S. X. Lin, Y. Kulizhskaya, C. Chelba, S. Vasudevan, E. Collins, V. Bashlovkina, T. Lu, D. Fritz, J. Park, Y. Zhou, C. Su, R. Tanburn, M. Sushkov, M. Rasquinha, J. Li, J. Prendki, Y. Li, P. LV, S. Sharma, H. Fitoussi, H. Huang, A. Dai, P. Dao, M. Burrows, H. Prior, D. Qin, G. Pundak, L. L. Sjoesund, A. Khurshudov, Z. Zhu, A. Webson, E. Kemp, T. Tan, S. Agrawal, S. Sargsyan, L. Cheng, J. Stephan, T. Kwiatkowski, D. Reid, A. Byravan, A. H. Michael, N. Heess, L. Zhou, S. Goenka, V. Carpenter, A. Levskaya, B. Wang, R. Roberts, R. Leblond, S. Chikkerur, S. Ginzburg, M. Chang, R. Riachi, Chuqiao, Xu, Z. Borsos, M. Pliskin, J. Pawar, M. Lustman, H. Kirkwood, A. Anand, A. Chaudhary, N. Kalb, K. Milan, S. Augenstein, A. Goldie, L. Prince, K. Raman, Y. Sun, V. Xia, A. Cohen, Z. Huo, J. Camp, S. Ellis, L. Zilka, D. V. Torres, L. Patel, S. Arora, B. Chan, J. Adler, K. Ayoub, J. Liang, F. Jamil, J. Jiang, S. Baumgartner, H. Sun, Y. Karov, Y. Akulov, H. Zheng, I. Cai, C. Fantacci, J. Rubin, A. R. Acha, M. Wang, N. D'Souza, R. Sathyanarayana, S. Dai, S. Rowe, A. Simanovsky, O. Goldman, Y. Kuang, X. Pan, A. Rosenberg, T. Rojas-Esponda, P. Dutta, A. Zeng, I. Jurenka, G. Farquhar, Y. Bansal, S. Iqbal, B. Roelofs, G.-Y. Joung, P. Beak, C. Ryu, R. Poplin, Y. Wu, J.-B. Alayrac, S. Buthpitiya, O. Ronneberger, C. Habtegebriel, W. Li, P. Cavallaro, A. Wei, G. Bensusky, T. Denk, H. Ganapathy, J. Stanway, P. Joshi, F. Bertolini, J. Lo, O. Ma, Z. Charles, G. Sampemane, H. Sahni, X. Chen, H. Askham, D. Gaddy, P. Young, J. Tan, M. Eyal, A. Bražinskas, L. Zhong, Z. Wu, M. Epstein, K. Bailey, A. Hard, K. Lee, S. Goldstein, A. Ruiz, M. Badawi, M. Lochbrunner, J. Kearns, A. Brown, F. Pardo, T. Weber, H. Yang, P.-P. Jiang, B. Akin, Z. Fu, M. Wainwright, C. Zou, M. Gaba, P.-A. Manzagol, W. Kan, Y. Song, K. Zainullina, R. Lin, J. Ko, S. Deshmukh, A. Jindal, J. Svensson, D. Tyam, H. Zhao, C. Kaeser-Chen, S. Baird, P. Moradi, J. Hall, Q. Guo, V. Tsang, B. Liang, F. Pereira, S. Ganesh, I. Korotkov, J. Adamek, S. Thiagarajan, V. Tran, C. Chen, C. Tar, S. Jain, I. Dasgupta, T. Bilal, D. Reitter, K. Zhao, G. Vezzani, Y. Gehman, P. Mehta, L. Beltrone, X. Dotiwalla, S. Guadarrama, Z. Abbas, S. Karp, P. Georgiev, C.-S. Ferng, M. Brockschmidt, L. Peng, C. Hirschall, V. Verma, Y. Bi, Y. Xiao, A. Dabush, K. Xu, P. Wallis, R. Parker, Q. Wang, Y. Xu, I. Safarli, D. Tewari, Y. Zhang, S. Kim, A. Gesmundo, M. Thomas, S. Levi, A. Chowdhury, K. Rao, P. Garst, S. Conway-Rahman, H. Ran, K. McKinney, Z. Xiao, W. Yu, R. Agrawal, A. Stjerngren, C. Ionescu, J. Chen, V. Sharma, J. Chiu, F. Liu, K. Franko, C. Sanford, X. Cai, P. Michel, S. Ganapathy, J. Labanowski, Z. Garrett, B. Vargas, S. Sun, B. Gale, T. Buschmann, G. Desjardins, N. Ghelani, P. Jain, M. Verma, C. Asawaroengchai, J. Eisenschlos, J. Harlalka, H. Kazawa, D. Metzler, J. Howland, Y. Jian, J. Ades, V. Shah, T. Gangwani, S. Lee, R. Ring, S. M. Hernandez, D. Reich, A. Sinha, A. Sathe, J. Kovac, A. Gill, A. Kannan, A. D'olimpio, M. Sevenich, J. Whang, B. Kim, K. C. Sim, J. Chen, J. Zhang, S. Lall, Y. Matias, B. Jia, A. Friesen, S. Nasso, A. Thapliyal, B. Perozzi, T. Yu, A. Shekhawat, S. Huda, P. Grabowski, E. Wang, A. Sreevatsa, H. Dib, M. Hassen, P. Schuh, V. Milutinovic, C. Welty, M. Quinn, A. Shah, B. Wang, G. Barth-Maron, J. Frye, N. Axelsson, T. Zhu, Y. Ma, I. Giannoumis, H. Sedghi, C. Ye, Y. Luan, K. Aydin, B. Chandra, V. Sampathkumar, R. Huang, V. Lavrenko, A. Eleryan, Z. Hong, S. Hansen, S. M. Carthy, B. Samanta, D. Čevd, X. Wang, F. Li, M. Voznesensky, M. Hoffman, A. Terzis, V. Sehwag, G. Fidel, L. He, M. Cai, Y. He, A. Feng, M. Nikoltchev, S. Phatale, J. Chase, R. Lawton, M. Zhang, T. Ouyang, M. Tragut, M. H. Manshadi, A. Narayanan, J. Shen, X. Gao, T. Bolukbasi, N. Roy, X. Li, D. Golovin, L. Panait, Z. Qin, G. Han, T. Anthony, S. Kudugunta, V. Patraucean, A. Ray, X. Chen, X. Yang, T. Bhatia, P. Talluri, A. Morris, A. Ražnatović, B. Brownfield, J. An, S. Peng, P. Kane, C. Zheng, N. Duduta, J. Kessinger, J. Noraky, S. Liu, K. Rong, P. Veličković, K. Rush, A. Goldin, F. Wei, S. M. R. Garlapati, G. Tofar, O. Kwon, J. Ni, E. Noland, J. D. Trapani, F. Beaufays, A. G. Roy, Y. Chow, A. Turker, G. Cideron, L. Mei, J. Clark, Q. Dou, M. Bošnjak, R. Leith, Y. Du, A. Yazdanbakhsh, M. Nasr, C. Kwak, S. S. Sheth, A. Kaskasoli, A. Anand, B. Lakshminarayanan, S. Jerome, D. Bieber, C.-T. Chu, A. Seneg, T. Shen, M. Sridhar, N. Ndebele, B. Beyret, S. Mohamed, M. Chen, M. Freitag, J. Guo, L. Liu, P. Roit, H. Chen, S. Yan, T. Stone, J. Co-Reyes, J. Cole, S. Scellato, S. Azizi, H. Hashemi, A. Jin, A. Iyer, M. Valentine, A. György, A. Ahuja, D. H. Diaz, C.-Y. Lee, N. Clement, W. Kong, D. Garmon, I. Watts, K. Bhatia, K. Gupta, M. Mieczkowski, H. Vallet, A. Taly, E. Loper, S. Joshi, J. Atwood, J. Chick, M. Collier, F. Iliopoulos, R. Trostle, B. Gunel, R. Leal-Cavazos, A. M. Hrafnkelsson, M. Guzman, X. Ju, A. Forbes, J. Emond, K. Chauhan, B. Caine, L. Xiao, W. Xeng, A. Moufarek, D. Murphy, M. Meng, N. Gupta, F. Riedel, A. Das, E. Lawal, S. Narayan, T. Sosea, J. Swirhun, L. Friso, B. Neyshabur, J. Lu, S. Girgin, M. Wunder, E. Yvinec, A. Pyne, V. Carbone, S. Rijhwani, Y. Guo, T. Doshi, A. Briukhov, M. Bain, A. Hitron, X. Wang, A. Gupta, K. Chen, C. Du, W. Zhang, D. Shah, A. Akula, M. Dylla, A. Kachra, W. Kuo, T. Zou, L. Wang, L. Xu, J. Zhu, J. Snyder, S. Menon, O. Firat, I. Mordatch, Y. Yuan, N. Ponomareva, R. Blevins, L. Moore, W. Wang, P. Chen, M. Scholz, A. Dwornik, J. Lin, S. Li, D. Antognini, T. I. X. Song, M. Miller, U. Kalra, A. Raveret, O. Akerlund, F. Wu, A. Nystrom, N. Godbole, T. Liu, H. DeBalsi, J. Zhao, B. Liu, A. Caciularu, L. Lax, U. Khandelwal, V. Langston, E. Bailey, S. Lattanzi, Y. Wang, N. Kovelamudi, S. Mondal, G. Guruganesh, N. Hua, O. Roval, P. Wesolowski, R. Ingale, J. Halcrow, T. Sohn, C. Angermueller, B. Raad, E. Stickgold, E. Lu, A. Kosik, J. Xie, T. Lillicrap, A. Huang, L. L. Zhang, D. Paulus, C. Farabet, A. Wertheim, B. Wang, R. Joshi, C. ling Ko, Y. Wu, S. Agrawal, L. Lin, X. Sheng, P. Sung, T. Breland-King, C. Butterfield, S. Gawde, S. Singh, Q. Zhang, R. Apte, S. Shetty, A. Hutter, T. Li, E. Salesky, F. Lebron, J. Kanerva, M. Paganini, A. Nguyen, R. Vallu, J.-T. Peter, S. Velury, D. Kao, J. Hoover, A. Bortsova, C. Bishop, S. Jakobovits, A. Agostini, A. Agarwal, C. Liu, C. Kwong, S. Tavakkol, I. Bica, A. Greve, A. GP, J. Marcus, L. Hou, T. Duerig, R. Moroshko, D. Lacey, A. Davis, J. Amelot, G. Wang, F. Kim, T. Strinopoulos, H. Wan, C. L. Lan, S. Krishnan, H. Tang, P. Humphreys, J. Bai, I. H. Shtacher, D. Machado, C. Pang, K. Burke, D. Liu, R. Aravamudhan, Y. Song, E. Hirst, A. Singh, B. Jou, L. Bai, F. Piccinno, C. K. Fu, R. Alazard, B. Meiri, D. Winter, C. Chen, M. Zhang, J. Heitkaemper, J. Lambert, J. Lee, A. Frömmgen, S. Rogulenko, P. Nair, P. Niemczyk, A. Bulynov, B. Xu, H. Shemtov, M. Zadimoghaddam, S. Toropov, M. Wirth, H. Dai, S. Gollapudi, D. Zheng, A. Kurakin, C. Lee, K. Bullard, N. Serrano, I. Balazevic, Y. Li, J. Schalkwyk, M. Murphy, M. Zhang, K. Sequeira, R. Datta, N. Agrawal, C. Sutton, P. Niemczyk, M. Chiang, W. Farhan, G. Thornton, K. Lin, T. Choma, H. Nguyen, K. Dasgupta, D. Robinson, I. Comşa, M. Riley, A. Pillai, B. Mustafa, M. Golan, A. Zandieh, J.-B. Lespiau, B. Porter, D. Ross, S. Rajayogam, M. Agarwal, S. Venugopalan, B. Shahriari, Q. Yan, H. Xu, T. Tobin, P. Dubov, H. Shi, A. Recasens, A. Kovsharov, S. Borgeaud, L. Dery, S. Vasanth, E. Gribovskaya, L. Qiu, M. Mahdih, W. Skut, E. Nielsen,

- C. Zheng, A. Yu, C. G. Bostock, S. Gupta, A. Archer, C. Rawles, E. Davies, A. Svyatkovskiy, T. Tsai, Y. Halpern, C. Reisswig, B. Wydrowski, B. Chang, J. Puigcerver, M. H. Taege, J. Li, E. Schnider, X. Li, D. Dena, Y. Xu, U. Telang, T. Shi, H. Zen, K. Kastner, Y. Ko, N. Subramaniam, A. Kumar, P. Blois, Z. Dai, J. Wieting, Y. Lu, Y. Zeldes, T. Xie, A. Hauth, A. Tifrea, Y. Li, S. El-Husseini, D. Abolafia, H. Zhou, W. Ding, S. Ghalebikesabi, C. Guia, A. Maksai, Ágoston Weisz, S. Arik, N. Sukhanov, A. Świątlik, X. Jia, L. Yu, W. Wang, M. Brand, D. Bloxwich, S. Kirmani, Z. Chen, A. Go, P. Sprechmann, N. Kannen, A. Carin, P. Sandhu, I. Edkins, L. Nooteboom, J. Gupta, L. Maggiore, J. Azizi, Y. Pritch, P. Yin, M. Gupta, D. Tarlow, D. Smith, D. Ivanov, M. Babaeizadeh, A. Goel, S. Kambala, G. Chu, M. Kastelic, M. Liu, H. Soltan, A. Stone, S. Agrawal, M. Kim, K. Soparkar, S. Tadepalli, O. Bunyan, R. Soh, A. Kannan, D. Kim, B. J. Chen, A. Halumi, S. Roy, Y. Wang, O. Sercinoglu, G. Gibson, S. Bhatnagar, M. Sano, D. von Dincklage, Q. Ren, B. Mitrevski, M. Olšák, J. She, C. Doersch, Jilei, Wang, B. Liu, Q. Tan, T. Yakar, T. Warkentin, A. Ramirez, C. Lebsack, J. Dillon, R. Mathews, T. Cogley, Z. Wu, Z. Chen, J. Simon, S. Nath, T. Sainath, A. Bendebury, R. Julian, B. Mankalale, D. Čurko, P. Zaccchello, A. R. Brown, K. Sodhia, H. Howard, S. Caelles, A. Gupta, G. Evans, A. Bulanova, L. Katzen, R. Goldenberg, A. Tsitsulin, J. Stanton, B. Schillings, V. Kovalev, C. Fry, R. Shah, K. Lin, S. Upadhyay, C. Li, S. Radpour, M. Maggioni, J. Xiong, L. Haas, J. Brennan, A. Kamath, N. Savinov, A. Nagnani, T. Yacovone, R. Kappedal, K. Andriopoulos, L. Lao, Y. Li, G. Rozhdestvenskiy, K. Hashimoto, A. Audibert, S. Austin, D. Rodriguez, A. Ruoss, G. Honke, D. Karkhanis, X. Xiong, Q. Wei, J. Huang, Z. Leng, V. Premachandran, S. Bileschi, G. Evangelopoulos, T. Mensink, J. Pavagadhi, D. Teplyashin, P. Chang, L. Xue, G. Tanzer, S. Goldman, K. Patel, S. Li, J. Wiesner, I. Zheng, I. Stewart-Binks, J. Han, Z. Li, L. Luo, K. Lenc, M. Lučić, F. Xue, R. Mullins, A. Guseynov, C.-C. Chang, I. Galatzer-Levy, A. Zhang, G. Bingham, G. Hu, A. Hartman, Y. Ma, J. Griffith, A. Irpan, C. Radebaugh, S. Yue, L. Fan, V. Ungureanu, C. Sorokin, H. Teufel, P. Li, R. Anil, D. Paparas, T. Wang, C.-C. Lin, H. Peng, M. Shum, G. Petrovic, D. Brady, R. Nguyen, K. Macherey, Z. Li, H. Singh, M. Yenugula, M. Iinuma, X. Chen, K. Kopparapu, A. Stern, S. Dave, C. Thekkath, F. Perot, A. Kumar, F. Li, Y. Xiao, M. Bilotti, M. H. Bateni, I. Noble, L. Lee, A. Vázquez-Reina, J. Salazar, X. Yang, B. Wang, E. Gruzewska, A. Rao, S. Raghuram, Z. Xu, E. Ben-David, J. Mei, S. Dalmia, Z. Zhang, Y. Liu, G. Bansal, H. Pankov, S. Schwarcz, A. Burns, C. Chan, S. Sanghai, R. Liang, E. Liang, A. He, A. Stuart, A. Narayanan, Y. Zhu, C. Frank, B. Fatemi, A. Sabne, O. Lang, I. Bhattacharya, S. Settle, M. Wang, B. McMahan, A. Tacchetti, L. B. Soares, M. Hadian, S. Cabi, T. Chung, N. Putikhin, G. Li, J. Chen, A. Tarango, H. Michalewski, M. Kazemi, H. Masoom, H. Sheftel, R. Shivanna, A. Vadali, R. Comanescu, D. Reid, J. Moore, A. Neelakantan, M. Sander, J. Herzig, A. Rosenberg, M. Dehghani, J. Choi, M. Fink, R. Hayes, E. Ge, S. Weng, C.-H. Ho, J. Karro, K. Krishna, L. N. Thiet, A. Skerry-Ryan, D. Eppens, M. Andreetto, N. Sarma, S. Bonacina, B. K. Ayan, M. Nawhal, Z. Shan, M. Dusenberry, S. Thakoor, S. Gubbi, D. D. Nguyen, R. Tsarfaty, S. Albanie, J. Mitrović, M. Gandhi, B.-J. Chen, A. Epasto, G. Stephanov, Y. Jin, S. Gehman, A. Amini, J. Weber, F. Behbahani, S. Xu, M. Allamanis, X. Chen, M. Ott, C. Sha, M. Jastrzebski, H. Qi, D. Greene, X. Wu, A. Toki, D. Vlasic, J. Shapiro, R. Kotikalapudi, Z. Shen, T. Saeki, S. Xie, A. Cassirer, S. Bharadwaj, T. Kiyono, S. Bhojanapalli, E. Rosenfeld, S. Ritter, J. Mao, J. G. Oliveira, Z. Egyed, B. Bandemer, E. Parisotto, K. Kinoshita, J. Pluto, P. Maniatis, S. Li, Y. Guo, G. Ghiasi, J. Tarbouriech, S. Chatterjee, J. Jin, Katrina, Xu, J. Palomaki, S. Arnold, M. Sewak, F. Piccinini, M. Sharma, B. Albrecht, S. Purser-haskell, A. Vaswani, C. Chen, M. Wisniewski, Q. Cao, J. Aslanides, N. M. Phu, M. Sieb, L. Agubuzu, A. Zheng, D. Sohn, M. Selvi, A. Andreassen, K. Subudhi, P. Erubetne, O. Woodman, T. Mery, S. Krause, X. Ren, X. Ma, J. Luo, D. Chen, W. Fan, H. Griffiths, C. Schuler, A. Li, S. Zhang, J.-M. Sarr, S. Luo, R. Patana, M. Watson, D. Naboulsi, M. Collins, S. Sidhwani, E. Hoogetboom, S. Silver, E. Caveness, X. Zhao, M. Rodriguez, M. Deines, L. Bai, P. Griffin, M. Tagliasacchi, E. Xue, S. R. Babbula, B. Pang, N. Ding, G. Shen, E. Peake, R. Crocker, S. S. Raghvendra, D. Swisher, W. Han, R. Singh, L. Wu, V. Pchelin, T. Munkhdalai, D. Alon, G. Bacon, E. Robles, J. Bulian, M. Johnson, G. Powell, F. T. Ferreira, Y. Li, F. Benzing, M. Velimirović, H. Soyer, W. Kong, Tony, Nguyễn, Z. Yang, J. Liu, J. van Amersfoort, D. Gillick, B. Sun, N. Rauschmayr, K. Zhang, S. Zhan, T. Zhou, A. Frolov, C. Yang, D. Vnukov, L. Rouillard, H. Li, A. Mandhane, N. Fallen, R. Venkataraman, C. H. Hu, J. Brennan, J. Lee, J. Chang, M. Sundermeyer, Z. Pan, R. Ke, S. Tong, A. Fabrikant, W. Bono, J. Gu, R. Foley, Y. Mao, M. Delakis, D. Bhaswar, R. Frostig, N. Li, A. Zipori, C. Hope, O. Kozlova, S. Mishra, J. Djolonga, C. Schiff, M. A. Mery, E. Briakou, P. Morgan, A. Wan, A. Hassidim, R. Skerry-Ryan, K. Sengupta, M. Jasarevic, P. Kallakuri, P. Kunkle, H. Brennan, T. Lieber, H. Mansoor, J. Walker, B. Zhang, A. Xie, G. Žužić, A. Chukwuka, A. Drusinsky, D. Cho, R. Yao, F. Naeem, S. Butt, E. Kim, Z. Jia, M. Jordan, A. Lelkes, M. Kurzeja, S. Wang, J. Zhao, A. Over, A. Chakladar, M. Prasetya, N. Jha, S. Ganapathy, Y. Cong, P. Shroff, C. Saroufim, S. Miryosefi, M. Hammad, T. Nasir, W. Xi, Y. Gao, Y. Maeng, B. Hora, C.-Y. Cheng, P. Hagani, Y. Lewenberg, C. Lu, M. Matysiak, N. Raisinghani, H. Wang, L. Baugher, R. Sukthankar, M. Giang, J. Schultz, N. Fiedel, M. Chen, C.-C. Lee, T. Dey, H. Zheng, S. Paul, C. Smith, A. Ly, Y. Wang, R. Bansal, B. Perz, S. Ricco, S. Blank, V. Keshava, D. Sharma, M. Chow, K. Lad, K. Jalan, S. Osindero, C. Swanson, J. Scott, A. Ilić, X. Li, S. R. Jonnalagadda, A. S. Soudagar, Y. Xiong, B.-O. Batsaikhan, D. Jarrett, N. Kumar, M. Shah, M. Lawlor, A. Waters, M. Graham, R. May, S. Ramos, S. Lefdal, Z. Cankara, N. Cano, B. O'Donoghue, J. Borovik, F. Liu, J. Grimstad, M. Alnahlawi, K. Tsihas, T. Hudson, N. Grigorev, Y. Jia, T. Huang, T. P. Igwe, S. Lebedev, X. Tang, I. Krivokon, F. Garcia, M. Tan, E. Jia, P. Stys, S. Vashishth, Y. Liang, B. Venkatraman, C. Gu, A. Kementsidis, C. Zhu, J. Jung, Y. Bai, M. J. Hosseini, F. Ahmed, A. Gupta, X. Yuan, S. Ashraf, S. Nigam, G. Vasudevan, P. Awasthi, A. M. Gilady, Z. Mariet, R. Eskander, H. Li, H. Hu, G. Garrido, P. Schlattner, G. Zhang, R. Saxena, P. Devič, K. Muralidharan, A. Murthy, Y. Zhou, M. Choi, A. Wongpanich, Z. Wang, P. Shah, Y. Xu, Y. Huang, S. Spencer, A. Chen, J. Cohan, J. Wang, J. Tompson, J. Wu, R. Haroun, H. Li, B. Huergo, F. Yang, T. Yin, J. Wendt, M. Bendersky, R. Chaabouni, J. Snider, J. Ferret, A. Jindal, T. Thompson, A. Xue, W. Bishop, S. M. Phal, A. Sharma, Y. Sung, P. Radhakrishnan, M. Shomrat, R. Ingle, R. Vij, J. Gilmer, M. D. Istin, S. Sobell, Y. Lu, E. Nottage, D. Sadigh, J. Willcock, T. Zhang, S. Xu, S. Brown, K. Lee, G. Wang, Y. Zhu, Y. Tay, C. Kim, A. Gutierrez, A. Sharma, Y. Xian, S. Seo, C. Cui, E. Pochernina, C. Baetu, K. Jastrzebski, M. Ly, M. Elhawaty, D. Suh, E. Sezener, P. Wang, N. Yuen, G. Tucker, J. Cai, Z. Yang, C. Wang, A. Muzio, H. Qian, J. Yoo, D. Lockhart, K. R. McKee, M. Guo, M. Mehrotra, A. Mendonça, S. V. Mehta, S. Ben, C. Tekur, J. Mu, M. Zhu, V. Krakovna, H. Lee, A. Maschinot, S. Cevey, H. Choe, A. Bai, H. Srinivasan, D. Gasaway, N. Young, P. Siegler, D. Holtmann-Rice, V. Piratla, K. Baumli, R. Yoge, A. Hofer, H. van Hasselt, S. Grant, Y. Chervonyi, D. Silver, A. Hogue, A. Agarwal, K. Wang, P. Singh, F. Flynn, J. Lipschultz, R. David, L. Bellot, Y.-Y. Yang, L. Le, F. Graziano, K. Olszewska, K. Hui, A. Maurya, N. Parotsidis, W. Chen, T. Oguntibi, J. Kelley, A. Baddepudi, J. Maurer, G. Shaw, A. Siegmán, L. Yang, S. Shetty, S. Roy, Y. Song, W. Stokowiec, R. Burnell, O. Savant, R. Busa-Fekete, J. Miao, S. Ghosh, L. MacDermed, P. Lippe, M. Dekhtarev, Z. Behrman, F. Mentzer, K. Nguyen, M. Wei, S. Verma, C. Knutsen, S. Dasari, Z. Yan, P. Mitrichev, X. Wang, V. Shejwalkar, J. Austin, S. Sunkara, N. Potti, Y. Virin, C. Wright, F. Liu, O. Riva, E. Pot, G. Kochanski, Q. Le, G. Balasubramaniam, A. Dhar, Y. Liao, A. Bloniarz, D. Shukla, E. Cole, J. Lee, S. Zhang, S. Kafle, S. Vashishtha, P. Mahmoudieh, G. Chen, R. Hoffmann, P. Srinivasan, A. D. Lago, Y. B. Shalom, Z. Wang, M. Elabd, A. Sharma, J. Oh, S. Kothawade, M. Le, M. Monteiro, S. Yang, K. Alarakya, R. Geirhos, D. Mincu, H. Garnes, H. Kobayashi, S. Mariooryad, K. Krasowiak, Zhixian, Lai, S. Mourad, M. Wang, F. Bu, O. Aharoni, G. Chen, A. Goyal, V. Zubov, A. Bapna, E. Dabir, N. Kothari, K. Lamerigts, N. D. Cao, J. Shar, C. Yew, N. Kulkarni, D. Mahaarachchi, M. Joshi, Z. Zhu, J. Lichtarge, Y. Zhou, H. Muckenhirn, V. Selo, O. Vinyals, P. Chen, A. Brohan, V. Mehta, S. Cogan, R. Wang, T. Geri, W.-J. Ko, W. Chen, F. Viola, K. Shivam, L. Wang, M. C. Elish, R. A. Popa, S. Pereira, J. Liu, R. Koster, D. Kim, G. Zhang, S. Ebrahimi, P. Talukdar, Y. Zheng, P. Poklukur, A. Mikhalap, D. Johnson, A. Vijayakumar, M. Omerick, M. Dibb, A. Dubey, Q. Hu, A. Suman, V. Aggarwal, I. Kornakov, F. Xia, W. Lowe, A. Kolganov, T. Xiao, V. Nikolaev, S. Hemingray, B. Li, J. Iljazi, M. Rybiński, B. Sandhu, P. Lu, T. Luong, R. Jenatton, V. Govindaraj, Hui, Li, G. Dulac-Arnold, W. Park, H. Wang, A. Modi, J. Pouget-Abadie, K. Greller, R. Gupta, R. Berry, P. Ramachandran, J. Xie, L. McCafferty, J. Wang, K. Gupta, H. Lim, B. Bratanić, A. Brock, I. Akolzin, J. Sproch, D. Karliner,

D. Kim, A. Goedeckemeyer, N. Shazeer, C. Schmid, D. Calandriello, P. Bhatia, K. Choromanski, C. Montgomery, D. Dua, A. Ramalho, H. King, Y. Gao, L. Nguyen, D. Lindner, D. Pitta, O. Johnson, K. Salama, D. Ardila, M. Han, E. Farnese, S. Odoom, Z. Wang, X. Ding, N. Rink, R. Smith, H. T. Lehari, E. Cohen, N. Vats, T. He, P. Gopavarapu, A. Paszke, M. Patel, W. V. Gansbeke, L. Loher, L. Castro, M. Voitoich, T. von Glehn, N. George, S. Niklaus, Z. Eaton-Rosen, N. Rakićević, E. Jue, S. Perel, C. Zhang, Y. Bahat, A. Pouget, Z. Xing, F. Huot, A. Shenoy, T. Bos, V. Coriou, B. Richter, N. Noy, Y. Wang, S. Ontanon, S. Qin, G. Makarchuk, D. Hassabis, Z. Li, M. Sharma, K. Venkatesan, I. Kemaev, R. Daniel, S. Huang, S. Shah, O. Ponce, Warren, Chen, M. Faruqi, J. Wu, S. Andačić, S. Payrits, D. McDuff, T. Hume, Y. Cao, M. Tessler, Q. Wang, Y. Wang, I. Rendulic, E. Agustsson, M. Johnson, T. Lando, A. Howard, S. G. S. Padmanabhan, M. Daswani, A. Banino, M. Kilgore, J. Heek, Z. Ji, A. Caceres, C. Li, N. Kassner, A. Vlaskin, Z. Liu, A. Grills, Y. Hou, R. Sukkerd, G. Cheon, N. Shetty, L. Markeeva, P. Stanczyk, T. Iyer, Y. Gong, S. Gao, K. Gopalakrishnan, T. Blyth, M. Reynolds, A. Bhoopchand, M. Bilenko, D. Gharibian, V. Zayats, A. Faust, A. Singh, M. Ma, H. Jiao, S. Vijayanarasimhan, L. Aroyo, V. Yadav, S. Chakera, A. Kakarla, V. Meshram, K. Gregor, G. Botea, E. Senter, D. Jia, G. Kovacs, N. Sharma, S. Baur, K. Kang, Y. He, L. Zhuo, M. Kostelac, I. Laish, S. Peng, L. O'Bryan, D. Kasenberg, G. R. Rao, E. Leurent, B. Zhang, S. Stevens, A. Salazar, Y. Zhang, I. Lobov, J. Walker, A. Porter, M. Redshaw, H. Ke, A. Rao, A. Lee, H. Lam, M. Moffitt, J. Kim, S. Qiao, T. Koo, R. Dadashi, X. Song, M. Sundararajan, P. Xu, C. Kawamoto, Y. Zhong, C. Barbu, A. Reddy, M. Verzett, L. Li, G. Papamakarios, H. Klimczak-Plucińska, M. Cassin, K. Kavukcuoglu, R. Swavely, A. Vaucher, J. Zhao, R. Hemsley, M. Tschannen, H. Ge, G. Menghani, Y. Yu, N. Ha, W. He, X. Wu, M. Song, R. Sterneck, S. Zinke, D. A. Calian, A. Marsden, A. C. Ruiz, M. Hessel, A. Gueta, B. Lee, B. Farris, M. Gupta, Y. Li, M. Saleh, V. Misra, K. Xiao, P. Mendolicchio, G. Buttimore, V. Krayvanova, N. Nayakanti, M. Wiethoff, Y. Pande, A. Mirhoseini, N. Lao, J. Liu, Y. Hua, A. Chen, Y. Malkov, D. Kalashnikov, S. Gupta, K. Audhkhasi, Y. Zhai, S. Kopalle, P. Jain, E. Ofek, C. Meyer, K. Baatarsukh, H. Strejček, J. Qian, J. Freedman, R. Figueira, M. Sokolik, O. Bachem, R. Lin, D. Kharrat, C. Hidey, P. Xu, D. Duan, Y. Li, M. Ersoy, R. Everett, K. Cen, R. Santamaria-Fernandez, A. Taubenfeld, I. Mackinnon, L. Deng, P. Zablotskaia, S. Viswanadha, S. Goel, D. Yates, Y. Deng, P. Choy, M. Chen, A. Sinha, A. Mossin, Y. Wang, A. Szlam, S. Hao, P. K. Rubenstein, M. Toksoz-Exley, M. Aperghis, Y. Zhong, J. Ahn, M. Isard, O. Lacombe, F. Luisier, A. Anastasiou, Y. Kalley, U. Prabhu, E. Dunleavy, S. Bijwadia, J. Mao-Jones, K. Chen, R. Pasumarthi, E. Wood, A. Dostmohamed, N. Hurley, J. Simsa, A. Parrish, M. Pajarskas, M. Harvey, O. Skopek, Y. Kochinski, J. Rey, V. Rieser, D. Zhou, S. J. Lee, T. Acharya, G. Li, J. Jiang, X. Zhang, B. Gipson, E. Mahintorabi, M. Gelmi, N. Khajehnouri, A. Yeh, K. Lee, L. Matthey, L. Baker, T. Pham, H. Fu, A. Pak, P. Gupta, C. Vasconcelos, A. Sadovsky, B. Walker, S. Hsiao, P. Zochbauer, A. Marzoca, N. Velan, J. Zeng, G. Baechler, D. Riess, D. Jain, Y. Huang, L. Tao, J. Maggs, N. Levine, J. Schneider, E. Gemzer, S. Petit, S. Han, Z. Fisher, D. Zelle, C. Biles, E. Ie, A. Fadeeva, C. Liu, J. V. Franco, A. Collister, H. Zhang, R. Wang, R. Zhao, L. Kieliger, K. Shuster, R. Zhu, B. Gong, L. Chan, R. Sun, S. Basu, R. Zimmermann, J. Hayes, A. Bapna, J. Snoek, W. Yang, P. Datta, J. A. Abdallah, K. Kilgour, L. Li, S. Mah, Y. Jun, M. Rivière, A. Karmarkar, T. Spalink, T. Huang, L. Gonzalez, D.-H. Tran, A. Nowak, J. Palowitch, M. Chadwick, E. Taluis, H. Mehta, T. Sellam, P. Fränken, M. Nicosia, K. He, A. Kini, D. Amos, S. Basu, H. Jobe, E. Shaw, Q. Xu, C. Evans, D. Ikeda, C. Yan, L. Jin, L. Wang, S. Yadav, I. Labzovsky, R. Sampath, A. Ma, C. Schumann, A. Siddhant, R. Shah, J. Youssef, R. Agarwal, N. Dabney, A. Tonioni, M. Ambar, J. Li, I. Guyon, B. Li, D. Soergel, B. Fang, G. Karadzhov, C. Udrescu, T. Trinh, V. Raunak, S. Noury, D. Guo, S. Gupta, M. Finkelstein, D. Petek, L. Liang, G. Billock, P. Sun, D. Wood, Y. Song, X. Yu, T. Matejovicova, R. Cohen, K. Andra, D. D'Ambrosio, Z. Deng, V. Nallatamby, E. Songhori, R. Dangovski, A. Lampinen, P. Botadra, A. Hillier, J. Cao, N. Baddi, A. Kuncoro, T. Yoshino, A. Bhagatwala, M. Ranzato, R. Schaeffer, T. Liu, S. Ye, O. Sarvana, J. Nam, C. Kuang, I. Gao, J. Baek, S. Mittal, A. Wahid, A. Gergely, B. Ni, J. Feldman, C. Muir, P. Lamblin, W. Macherey, E. Dyer, L. Kilpatrick, V. Campos, M. Bhutani, S. Fort, Y. Ahmad, A. Severyn, K. Chatziprimou, O. Ferludin, M. Dimarco, A. Kusupati, J. Heyward, D. Bahir, K. Vilella, K. Millican, D. Marcus, S. Bahargam, C. Unlu, N. Roth, Z. Wei, S. Gopal, D. Ghoshal, E. Lee, S. Lin, J. Lees, D. Lee, A. Hosseini, C. Fan, S. Neel, M. Wu, Y. Altun, H. Cai, E. Piqueras, J. Woodward, A. Bissacco, S. Haykal, M. Bordbar, P. Sundaram, S. Hodkinson, D. Toyama, G. Polovets, A. Myers, A. Sinha, T. Levinboim, K. Krishnakumar, R. Chhaparia, T. Sholokhova, N. B. Gundavarapu, G. Jawahar, H. Qureshi, J. Hu, N. Momchev, M. Rahtz, R. Wu, A. P. S. K. Dhamdhare, M. Guo, U. Gupta, A. Eslami, M. Schain, M. Blokzijl, D. Welling, D. Orr, L. Bolelli, N. Perez-Nieves, M. Sirotenko, A. Prasad, A. Kar, B. D. B. Pigem, T. Terzi, G. Weisz, D. Ghosh, A. Mavalankar, D. Madeka, K. Dagaard, H. Adam, V. Shah, D. Berman, M. Tran, S. Baker, E. Andrejczuk, G. Chole, G. Raboshchuk, M. Mirzazadeh, T. Kagohara, S. Wu, C. Schallhart, B. Orlando, C. Wang, A. Rustemi, H. Xiong, H. Liu, A. Vezzer, N. Ramsden, S. yin Chang, S. Mudgal, Y. Li, N. Vieillard, Y. Hoshen, F. Ahmad, A. Slone, A. Hua, N. Potikha, M. Rossini, J. Stritar, S. Prakash, Z. Wang, X. Dong, A. Nazari, E. Nehoran, K. Tekelioglu, Y. Li, K. Badola, T. Funkhouser, Y. Li, V. Yerram, R. Ganeshan, D. Formoso, K. Langner, T. Shi, H. Li, Y. Yamamori, A. Panda, A. Saade, A. S. Scarpatti, C. Breaux, C. Carey, Z. Zhou, C.-J. Hsieh, S. Bridges, A. Butryna, N. Gupta, V. Tulsyan, S. Woo, E. Eltyshev, W. Grathwohl, C. Parks, S. Benjamin, R. Panigrahy, S. Dodhia, D. D. Freitas, C. Sauer, W. Song, F. Alet, J. Tolins, C. Paduraru, X. Zhou, B. Albert, Z. Zhang, L. Shu, M. Bansal, S. Nguyen, A. Globerson, O. Xiao, J. Manyika, T. Hennigan, R. Rong, J. Matak, A. Bakalov, A. Sharma, D. Sinopalnikov, A. Pierson, S. Roller, G. Brown, M. Gao, T. Fukuzawa, A. Ghafouri, K. Vassigh, I. Barr, Z. Wang, A. Korsun, R. Jayaram, L. Ren, T. Zaman, S. Khan, Y. Lunts, D. Deutsch, D. Uthus, N. Katz, M. Samsikova, A. Khalifa, N. Sethi, J. Sun, L. Tang, U. Alon, X. Luo, D. Yu, A. Nayyar, B. Petrini, W. Truong, V. Hellendoorn, N. Chinaev, C. Alberti, W. Wang, J. Hu, V. Mirrokni, A. Balashankar, A. Aharon, A. Mehta, A. Iscen, J. Kready, L. Manning, A. Mohananey, Y. Chen, A. Tripathi, A. Wu, I. Petrovski, D. Hwang, M. Bauml, S. Chandrakaladharan, Y. Liu, R. Coaguila, M. Chen, S. Ma, P. Tafti, S. Tatineni, T. Spitz, J. Ye, P. Vicol, M. Rosca, A. Puigdomènech, Z. Yahav, S. Ghemawat, H. Lin, P. Kirk, Z. Nabulis, S. Brin, B. Bohnet, K. Caluwaerts, A. S. Veerubhotla, D. Zheng, Z. Dai, P. Petrov, Y. Xu, R. Mehran, Z. Xu, L. Zintgraf, J. Choi, S. A. Hombaiah, R. Thoppilan, S. Reddi, L. Lew, L. Li, K. Webster, K. Sawhney, L. Lamprou, S. Shakeri, M. Lunayach, J. Chen, S. Bagri, A. Salcianu, Y. Chen, Y. Donchev, C. Magister, S. Nørly, V. Rodrigues, T. Izo, H. Noga, J. Zou, T. Köppe, W. Zhou, K. Lee, X. Long, D. Eisenbud, A. Chen, C. Schenck, C. M. To, P. Zhong, E. Taropa, M. Truong, O. Levy, D. Martins, Z. Zhang, C. Semturs, D. Zhang, A. Yakubovich, P. Moreno, L. McConnaughey, D. Lu, S. Redmond, L. Weerts, Y. Bitton, T. Refice, N. Lacasse, A. Conmy, C. Tallec, J. Odell, H. Forbes-Pollard, A. Socala, J. Hoeck, P. Kohli, A. Walton, R. Wang, M. Sazanovich, K. Zhu, A. Kapishnikov, R. Galt, M. Denton, B. Murdoch, C. Sikora, K. Mohamed, W. Wei, U. First, T. McConnell, L. C. Cobo, J. Qin, T. Avrahami, D. Balle, Y. Watanabe, A. Louis, A. Kraft, S. Ariafar, Y. Gu, E. Rives, C. Yoon, A. Rusu, J. Cobon-Kerr, C. Hahn, J. Luo, Yuvein, Zhu, N. Ahuja, R. Benenson, R. L. Kaufman, H. Yu, L. Hightower, J. Zhang, D. Ni, L. A. Hendricks, G. Wang, G. Yona, L. Jain, P. Barrio, S. Bhupatiraju, S. Velusamy, A. Dafee, S. Riedel, T. Thomas, Z. Yuan, M. Bellaiche, S. Panthaplackel, K. Kloboves, S. Jauhari, C. Akbulut, T. Davchev, E. Gladchenko, D. Madras, A. Chuklin, T. Hill, Q. Yuan, M. Madhavan, L. Leonhard, D. Scandinaro, Q. Chen, N. Niu, A. Douillard, B. Damoc, Y. Onoe, F. Pedregosa, F. Bertsch, C. Leichner, J. Pagadora, J. Malmud, S. Ponda, A. Twigg, O. Duzhyi, J. Shen, M. Wang, R. Garg, J. Chen, U. Evci, J. Lee, L. Liu, K. Kojima, M. Yamaguchi, A. Rajendran, A. Piergiovanni, V. K. Rajendran, M. Fornoni, G. Ibagon, H. Ragan, S. M. Khan, J. Blitzer, A. Bunner, G. Sun, T. Kosakai, S. Lundberg, N. Elue, K. Guu, S. Park, J. Park, A. Narayanaswamy, C. Wu, J. Mudigonda, T. Cohn, H. Mu, R. Kumar, L. Graesser, Y. Zhang, R. Killam, V. Zhuang, M. Giménez, W. A. Jishi, R. Ley-Wild, A. Zhai, K. Osawa, D. Cedillo, J. Liu, M. Upadhyay, M. Sieniek, R. Sharma, T. Paine, A. Angelova, S. Addepalli, C. Parada, K. Majumder, A. Lamp, S. Kumar, X. Deng, A. Myaskovsky, T. Sabolić, J. Dudek, S. York, F. de Chaumont Quiry, J. Nie, D. Cattle, A. Gunjan, B. Piot, W. Khawaja, S. Bang, S. Wang, S. Khodadadeh, R. R. P. Rawlani, R. Powell, K. Lee, J. Griesser, G. Oh, C. Magalhaes, Y. Li, S. Tokumine, H. N. Vogel, D. Hsu, A. BC, D. Jindal, M. Cohen, Z. Yang, J. Yuan, D. de Cesare, T. Bruguier, J. Xu, M. Roy, A. Jacovi, D. Belov,

- R. Arya, P. Meadowlark, S. Cohen-Ganor, W. Ye, P. Morris-Suzuki, P. Banzal, G. Song, P. Ponnuramu, F. Zhang, G. Scrivener, S. Zaiem, A. R. Rochman, K. Han, B. Ghazi, K. Lee, S. Drath, D. Suo, A. Girgis, P. Shenoy, D. Nguyen, D. Eck, S. Gupta, L. Yan, J. Carreira, A. Gulati, R. Sang, D. Mirylenka, E. Cooney, E. Chou, M. Ling, C. Fan, B. Coleman, G. Tubone, R. Kumar, J. Baldrige, F. Hernandez-Campos, A. Lazaridou, J. Besley, I. Yona, N. Bulut, Q. Wellens, A. Pierigiovanni, J. George, R. Green, P. Han, C. Tao, G. Clark, C. You, A. Abdolmaleki, J. Fu, T. Chen, A. Chaugule, A. Chandorkar, A. Rahman, W. Thompson, P. Koanantakool, M. Bernico, J. Ren, A. Vlasov, S. Vassilvitskii, M. Kula, Y. Liang, D. Kim, Y. Huang, C. Ye, D. Lepikhin, and W. Helmholtz, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," 2025. [Online]. Available: <https://arxiv.org/abs/2507.06261>
- [11] T. Dao, D. Haziza, F. Massa, and G. Sizov, "Flash-decoding for long-context inference," Online, 2023, <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>.
- [12] J. Dong, J. Rosenblum, and S. Narayanasamy, "Toleo: Scaling freshness to tera-scale memory using cxl and pim," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2025, p. 313–328. [Online]. Available: <https://doi.org/10.1145/36222781.3674180>
- [13] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnston, J. Saxe, J. Jia, K. V. Alwala, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, L. Rantala-Yearly, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhende, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. E. Tan, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Grattafiori, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Vaughan, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Franco, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Wyatt, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Ozgenel, F. Caggioni, F. Guzmán, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Thattai, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, I. Damla, I. Molybog, I. Tufanov, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Prasad, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Huang, K. Chawla, K. Lakhotia, K. Huang, L. Chen, L. Garg, L. A. L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabza, M. Avalani, M. Bhatt, M. Tsimpoukelli, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. P. Laptev, N. Dong, N. Zhang, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Li, R. Hogan, R. Battey, R. Wang, R. Maheswari, R. Howes, R. Rinott, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Kohler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Albiero, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wang, X. Wu, X. Wang, X. Xia, X. Wu, X. Gao, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Hao, Y. Qian, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, and Z. Zhao, "The llama 3 herd of models," 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [14] C. Giannoula, I. Fernandez, J. Gómez-Luna, N. Koziris, G. Goumas, and O. Mutlu, "Towards efficient sparse matrix vector multiplication on real processing-in-memory architectures," in *Abstract Proceedings of the 2022 ACM SIGMETRICS/IFIP PERFORMANCE Joint International Conference on Measurement and Modeling of Computer Systems*, ser. SIGMETRICS/PERFORMANCE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 33–34. [Online]. Available: <https://doi.org/10.1145/3489048.3522661>
- [15] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnston, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Yearly, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang,

- O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenheide, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Baeviski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damlaj, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Jagadeesh, K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A. L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabba, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait, Z. DeVito, Z. Rosenbick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, "The llama 3 herd of models," 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [16] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, "Pim is all you need: A cxl-enabled gpu-free system for large language model inference," in *2025 International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.
- [17] J. Gómez-Luna, I. E. Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System," *IEEE Access*, vol. 10, pp. 52 565–52 608, 2022.
- [18] J. He and J. Zhai, "Fastdecode: High-throughput gpu-efficient llm serving using heterogeneous pipelines," 2024. [Online]. Available: <https://arxiv.org/abs/2403.11421>
- [19] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. Vijaykumar, "Newton: A dram-maker's accelerator-in-memory (aim) architecture for machine learning," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 372–385.
- [20] Y. He, H. Mao, C. Giannoula, M. Sadrosadati, J. Gómez-Luna, H. Li, X. Li, Y. Wang, and O. Mutlu, "Papi: Exploiting dynamic parallelism in large language model decoding with a processing-in-memory-enabled computing system," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 766–782. [Online]. Available: <https://doi.org/10.1145/3676641.3716009>
- [21] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "Neupims: A npu-pim heterogeneous acceleration for batched inference of large language model," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2024, pp. 1–16.
- [22] S. Hong, S. Moon, J. Kim, S. Lee, M. Kim, D. Lee, and J.-Y. Kim, "Dfx: A low-latency multi-fpga appliance for accelerating transformer-based text generation," in *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '22. IEEE Press, 2023, p. 616–630. [Online]. Available: <https://doi.org/10.1109/MICRO56248.2022.00051>
- [23] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu *et al.*, "Gpipe: Easy scaling with micro-batch pipeline parallelism," *proceeding of Computer Science > Computer Vision and Pattern Recognition*, 2019.
- [24] B. Hyun, T. Kim, D. Lee, and M. Rhu, "Pathfinding Future PIM Architectures by Demystifying a Commercial PIM Technology," *arXiv*, 2023.
- [25] T. Jeong and E.-Y. Chung, "Pipepim: Maximizing computing unit utilization in ml-oriented digital pim by pipelining and dual buffering," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 12, pp. 4585–4598, 2024.
- [26] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," 2023. [Online]. Available: <https://arxiv.org/abs/2310.06825>
- [27] H. Kal, C. Yoo, and W. W. Ro, "Aespa: Asynchronous execution scheme to exploit bank-level parallelism of processing-in-memory," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 815–827.
- [28] B. Kim, S. Cha, S. Park, J. Lee, S. Lee, S.-h. Kang, J. So, K. Kim, J. Jung, J.-G. Lee, S. Lee, Y. Paik, H. Kim, J.-S. Kim, W.-J. Lee, Y. Ro, Y. Cho, J. H. Kim, J. Song, J. Yu, S. Lee, J. Cho, and K. Sohn, "The breakthrough memory solutions for improved performance on llm inference," *IEEE Micro*, vol. 44, no. 3, pp. 40–48, 2024.
- [29] B. J. Kim, T. Jeong, S. Yun, and E.-Y. Chung, "Dh-pim: Maximizing computing unit utilization in digital pim by dual half mode extension," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 1–1, 2025.
- [30] G. Kim, J. Kim, N. Kim, W. Shin, J. Won, H. Joo, H. Choi, B. An, G. Shin, D. Yun, J. Kim, C. Kim, I. Kim, J. Park, Y. Song, B. Yang, H. Lee, S. Park, W. Lee, S. Kim, Y. Park, Y. Jung, G.-H. Park, and E. Lim, "Sk hynix ai-specific computing memory solution: From aim device to heterogeneous aimx-xpu system for comprehensive llm inference," in *2024 IEEE Hot Chips 36 Symposium (HCS)*, 2024, pp. 1–26.
- [31] J. H. Kim, S.-H. Kang, S. Lee, H. Kim, Y. Ro, S. Lee, D. Wang, J. Choi, J. So, Y. Cho *et al.*, "Aquabolt-xl hbm2-pim, lpddr5-pim with in-memory processing, and axdim with acceleration buffer," *IEEE Micro*, vol. 42, no. 3, pp. 20–30, 2022.
- [32] J. H. Kim, Y. Ro, J. So, S. Lee, S.-h. Kang, Y. Cho, H. Kim, B. Kim, K. Kim, S. Park *et al.*, "Samsung pim/pnm for transformer based ai:

- Energy efficiency on pim/pnm cluster,” in *2023 IEEE Hot Chips 35 Symposium (HCS)*. IEEE Computer Society, 2023, pp. 1–31.
- [33] D. Kwon, S. Lee, K. Kim, S. Oh, J. Park, G.-M. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, V. Kornijcuk, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An, J. Lee, D. Ko, Y. Jun, I. Kim, C. Song, I. Kim, C. Park, S. Kim, C. Jeong, E. Lim, D. Kim, J. Jang, I. Park, J. Chun, and J. Cho, “A lynn 1.25v 8gb 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep learning application,” *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, pp. 291–302, 2023.
- [34] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [35] Y. Kwon, G. Kim, N. Kim, W. Shin, J. Won, H. Joo, H. Choi, B. An, G. Shin, D. Yun, J. Kim, C. Kim, I. Kim, J. Park, C. Park, Y. Song, B. Yang, H. Lee, S. Park, W. Lee, S. Lee, K. Kim, D. Kwon, C. Jeong, J. Kim, E. Lim, and J. Chun, “Memory-centric computing with sk hynix’s domain-specific memory,” in *2023 IEEE Hot Chips 35 Symposium (HCS)*, 2023, pp. 1–26.
- [36] Y. Kwon, K. Vladimir, N. Kim, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An *et al.*, “System architecture and software stack for gddr6-aim,” in *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE, 2022, pp. 1–25.
- [37] Y. Kwon, K. Vladimir, N. Kim, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An, J. Kim, J. Lee, I. Kim, J. Park, C. Park, Y. Song, B. Yang, H. Lee, S. Kim, D. Kwon, S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, M. Lee, M. Shin, M. Shin, J. Cha, C. Jung, K. Chang, C. Jeong, E. Lim, I. Park, J. Chun, and S. Hynix, “System architecture and software stack for gddr6-aim,” in *2022 IEEE Hot Chips 34 Symposium (HCS)*, 2022, pp. 1–25.
- [38] Y.-C. Kwon, S. H. Lee, J. Lee, S.-H. Kwon, J. M. Ryu, J.-P. Son, O. Seongil, H.-S. Yu, H. Lee, S. Y. Kim *et al.*, “25.4 a 20nm 6gb function-in-memory dram, based on hbm2 with a 1.2 tflops programmable computing unit using bank-level parallelism, for machine learning applications,” in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64. IEEE, 2021, pp. 350–352.
- [39] D. Lee, B. Hyun, T. Kim, and M. Rhu, “Pim-mm: A memory management unit for accelerating data transfers in commercial pim systems,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.06204>
- [40] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, K. Vladimir, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, J. Lee, D. Ko, Y. Jun, K. Cho, I. Kim, C. Song, C. Jeong, D. Kwon, J. Jang, I. Park, J. Chun, and J. Cho, “A lynn 1.25v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep-learning applications,” in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 1–3.
- [41] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin *et al.*, “Hardware architecture and software stack for pim based on commercial dram technology: Industrial product,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 43–56.
- [42] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, “Hardware architecture and software stack for pim based on commercial dram technology : Industrial product,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 43–56.
- [43] C. Li, Z. Zhou, Y. Wang, F. Yang, T. Cao, M. Yang, Y. Liang, and G. Sun, “Pim-dl: Expanding the applicability of commodity dram-pims for deep learning via algorithm-system co-optimization,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 879–896. [Online]. Available: <https://doi.org/10.1145/3620665.3640376>
- [44] C. Li, Z. Zhou, S. Zheng, J. Zhang, Y. Liang, and G. Sun, “Specpim: Accelerating speculative inference on pim-enabled system via architecture-dataflow co-exploration,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 950–965. [Online]. Available: <https://doi.org/10.1145/3620666.3651352>
- [45] D. Li*, R. Shao*, A. Xie, Y. Sheng, L. Zheng, J. E. Gonzalez, I. Stoica, X. Ma, and H. Zhang, “How long can open-source llms truly promise on context length?” June 2023. [Online]. Available: <https://lmsys.org/blog/2023-06-29-longchat>
- [46] Y. Liang, H. Shi, H. Shao, and Z. Wang, “Accllm: Accelerating long-context llm inference via algorithm-hardware co-design,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.03745>
- [47] T. Liu, C. Xu, and J. McAuley, “Repobench: Benchmarking repository-level code auto-completion systems,” *arXiv preprint arXiv:2306.03091*, 2023.
- [48] I. Meta Platforms, “The llama 4 herd: The beginning of a new era of natively multimodal ai innovation,” April 2025, released under the Llama 4 Community License Agreement. [Online]. Available: <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>
- [49] S. Moon, J. Cha, H. Park, and J.-Y. Kim, “Hybe: Gpu-npu hybrid system for efficient llm inference with million-token context window,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 808–820. [Online]. Available: <https://doi.org/10.1145/3695053.3731051>
- [50] MosaicML, “Introducing mpt-7b: A new standard for open-source, commercially usable llms,” 2023. [Online]. Available: www.mosaicml.com/blog/mpt-7b
- [51] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungrun, “A Modern Primer on Processing in Memory,” *arXiv*, 2020.
- [52] OpenAI, “Openai o1 system card,” <https://openai.com/index/openai-o1-system-card/>, 2024, accessed on: 2024-11-21.
- [53] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, R. Avila, I. Babuschkin, S. Balaji, V. Balcom, P. Baltescu, H. Bao, M. Bavarian, J. Belgum, I. Bello, J. Berdine, G. Bernadett-Shapiro, C. Berner, L. Bogdonoff, O. Boiko, M. Boyd, A.-L. Brakman, G. Brockman, T. Brooks, M. Brundage, K. Button, T. Cai, R. Campbell, A. Cann, B. Carey, C. Carlson, R. Carmichael, B. Chan, C. Chang, F. Chantzis, D. Chen, S. Chen, R. Chen, J. Chen, M. Chen, B. Chess, C. Cho, C. Chu, H. W. Chung, D. Cummings, J. Currier, Y. Dai, C. Decareaux, T. Degry, N. Deutsch, D. Deville, A. Dhar, D. Dohan, S. Dowling, S. Dunning, A. Ecoffet, A. Eleti, T. Eloundou, D. Farhi, L. Fedus, N. Felix, S. P. Fishman, J. Forte, I. Fulford, L. Gao, E. Georges, C. Gibson, V. Goel, T. Gogineni, G. Goh, R. Gontijo-Lopes, J. Gordon, M. Grafstein, S. Gray, R. Greene, J. Gross, S. S. Gu, Y. Guo, C. Hallacy, J. Han, J. Harris, Y. He, M. Heaton, J. Heidecke, C. Hesse, A. Hickey, W. Hickey, P. Hoeschele, B. Houghton, K. Hsu, S. Hu, X. Hu, J. Huizinga, S. Jain, S. Jain, J. Jang, A. Jiang, R. Jiang, H. Jin, D. Jin, S. Jomoto, B. Jonn, H. Jun, T. Kaftan, Łukasz Kaiser, A. Kamali, I. Kanitscheider, N. S. Keskar, T. Khan, L. Kilpatrick, J. W. Kim, C. Kim, Y. Kim, J. H. Kirchner, J. Kiros, M. Knight, D. Kokotajlo, Łukasz Kondraciuk, A. Kondrich, A. Konstantinidis, K. Kosic, G. Krueger, V. Kuo, M. Lampe, I. Lan, T. Lee, J. Leike, J. Leung, D. Levy, C. M. Li, R. Lim, M. Lin, S. Lin, M. Litwin, T. Lopez, R. Lowe, P. Lue, A. Makanju, K. Malfacini, S. Manning, T. Markov, Y. Markovski, B. Martin, K. Mayer, A. Mayne, B. McGrew, S. M. McKinney, C. McLeavey, P. McMillan, J. McNeil, D. Medina, A. Mehta, J. Menick, L. Metz, A. Mishchenko, P. Mishkin, V. Monaco, E. Morikawa, D. Mossing, T. Mu, M. Murati, O. Murk, D. Mély, A. Nair, R. Nakano, R. Nayak, A. Neelakantan, R. Ngo, H. Noh, L. Ouyang, C. O’Keefe, J. Pachocki, A. Paino, J. Palermo, A. Pantuliano, G. Parascandolo, J. Parish, E. Parparita, A. Passos, M. Pavlov, A. Peng, A. Perelman, F. de Avila Belbute Peres, M. Petrov, H. P. de Oliveira Pinto, Michael, Pokorny, M. Pokrass, V. H. Pong, T. Powell, A. Power, B. Power, E. Proehl, R. Puri, A. Radford, J. Rae, A. Ramesh, C. Raymond, F. Real, K. Rimbach, C. Ross, B. Rotsted, H. Roussez, N. Ryder, M. Saltarelli, T. Sanders, S. Santurkar, G. Sastry, H. Schmidt, D. Schnurr, J. Schulman, D. Selsam, K. Sheppard, T. Sherbakov, J. Shieh, S. Shoker, P. Shyam, S. Sidor, E. Sigler, M. Simens, J. Sitkin, K. Slama, I. Sohl, B. Sokolowsky, Y. Song, N. Staudacher, F. P. Such, N. Summers, I. Sutskever, J. Tang, N. Tezak, M. B. Thompson, P. Tillet, A. Tootoonchian, E. Tseng, P. Tuggle, N. Turley, J. Tworek, J. F. C. Uribe, A. Vallone, A. Vijayvergiya, C. Voss, C. Wainwright, J. J. Wang, A. Wang, B. Wang, J. Ward, J. Wei, C. Weinmann, A. Welihinda, P. Welinder, J. Weng, L. Weng,

- M. Wiethoff, D. Willner, C. Winter, S. Wolrich, H. Wong, L. Workman, S. Wu, J. Wu, M. Wu, K. Xiao, T. Xu, S. Yoo, K. Yu, Q. Yuan, W. Zaremba, R. Zellers, C. Zhang, M. Zhang, S. Zhao, T. Zheng, J. Zhuang, W. Zhuk, and B. Zoph, "Gpt-4 technical report," 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [54] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "Attacc! unleashing the power of pim for batched transformer-based generative model inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2024, pp. 1–17.
- [55] P. Patel, E. Choukse, C. Zhang, A. Shah, Íñigo Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative llm inference using phase splitting," 2024. [Online]. Available: <https://arxiv.org/abs/2311.18677>
- [56] Y. Qin, Y. Wang, Z. Zhao, X. Yang, Y. Zhou, S. Wei, Y. Hu, and S. Yin, "Mecla: Memory-compute-efficient llm accelerator with scaling sub-matrix partition," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 1032–1047.
- [57] P. G. Recasens, F. Agullo, Y. Zhu, C. Wang, E. K. Lee, O. Tardieu, J. Torres, and J. L. Berral, "Mind the memory gap: Unveiling gpu bottlenecks in large-batch llm inference," *arXiv preprint arXiv:2503.08311*, 2025.
- [58] G. Seo and S. Ryu, "Specboost: Accelerating tiled sparse matrix multiplication via dataflow speculation," *IEEE Access*, vol. 13, pp. 45 568–45 576, 2025.
- [59] M. Seo, X. T. Nguyen, S. J. Hwang, Y. Kwon, G. Kim, C. Park, I. Kim, J. Park, J. Kim, W. Shin, J. Won, H. Choi, K. Kim, D. Kwon, C. Jeong, S. Lee, Y. Choi, W. Byun, S. Baek, H.-J. Lee, and J. Kim, "Ianus: Integrated accelerator based on npu-pim unified memory system," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 545–560. [Online]. Available: <https://doi.org/10.1145/3620666.3651324>
- [60] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," *arXiv*, 2019.
- [61] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.
- [62] SK hynix, Memory Solution Product Design (MSPD) Division, "Aimx platform white paper," 2023. [Online]. Available: <https://product.skhynix.com/support/downloads/kits.go>
- [63] H. Son, G. Jonatan, X. Wu, H. Cho, K. Shivdikar, J. L. Abellán, A. Joshi, D. Kaeli, and J. Kim, "Pimnet: A domain-specific network for efficient collective communication in scalable pim," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1557–1572.
- [64] M. Stephenson and R. Rangan, "Pgz: automatic zero-value code specialization," in *Proceedings of the 30th ACM SIGPLAN International Conference on Compiler Construction*, ser. CC 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 36–46. [Online]. Available: <https://doi.org/10.1145/3446804.3446845>
- [65] G. Team, P. Georgiev, V. I. Lei, R. Burnell, L. Bai, A. Gulati, G. Tanzer, D. Vincent, Z. Pan, S. Wang *et al.*, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," *arXiv preprint arXiv:2403.05530*, 2024.
- [66] Q. Team, "Qwen3-max: Just scale it," September 2025.
- [67] The IREE Authors, "Iree," <https://iree.dev/>, 2019, an MLIR-based compiler and runtime for ML models from multiple frameworks.
- [68] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in neural information processing systems*, 2017.
- [69] T. Wang, Z. Shen, and Z. Shao, "Cnn acceleration with joint optimization of practical pim and gpu on embedded devices," in *2022 IEEE 40th International Conference on Computer Design (ICCD)*, 2022, pp. 377–384.
- [70] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, "Qwen3 technical report," 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [71] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, "Flashinfer: Efficient and customizable attention engine for llm inference serving," *arXiv preprint arXiv:2501.01005*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.01005>
- [72] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-Based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/yu>
- [73] T. Yuan, X. Ning, D. Zhou, Z. Yang, S. Li, M. Zhuang, Z. Tan, Z. Yao, D. Lin, B. Li, G. Dai, S. Yan, and Y. Wang, "Lv-eval: A balanced long-context benchmark with 5 length levels up to 256k," 2024.
- [74] M. Zhong, D. Yin, T. Yu, A. Zaidi, M. Mutuma, R. Jha, A. H. Awadallah, A. Celikyilmaz, Y. Liu, X. Qiu, and D. Radev, "QMSum: A new benchmark for query-based multi-domain meeting summarization," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, and Y. Zhou, Eds. Online: Association for Computational Linguistics, Jun. 2021, pp. 5905–5921. [Online]. Available: <https://aclanthology.org/2021.naacl-main.472>
- [75] M. Zhou, W. Xu, J. Kang, and T. Rosing, "TransPIM: A Memory-based Acceleration via Software-Hardware Co-Design for Transformer," *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, vol. 00, pp. 1071–1085, 2022.